

FastCAE-OpenFoam 集成示例

（可视化集成与插件拓展集成）

FastCAE 研发小组

2020 年 03 月 25 日

1. 概述

1.1 案例介绍

本案例结合 FastCAE 提供的可视化定制功能与插件拓展机制，封装了 simpleFoam 求解器，形成了一款针对不可压缩流体稳态仿真问题的软件。本软件具有友好的交互界面，与求解器无缝对接，形成集前后处理于一体的针对特定问题的专用 CAE 仿真软件。

本案例名称为 OpenFoam 集成案例，该案例所有代码可以在网站 https://pan.baidu.com/s/1GW5cGOykpQJ5jA_CMtQ_SQ（提取码：c295）处下载。

1.2 集成目标

- 通过 FastCAE2.0 集成框架完成一个典型的以 OpenFoam 软件为求解器的软件产品集成工作，实现专业的用户界面定制，便捷求解操作。
- 通过本案例的开发，能够使用户快速掌握 FastCAE2.0 插件开发与定制开发相结合的开发过程，为 OpenFoam 软件集成与二次开发提供指导。

1.3 环境说明

开发环境

- Ubuntu16.04

- QtCreator3.4.1
- Linux 下 qt-opensource-linux-x64-5.4.2.run
- FastCAE 2.0
- OpenFoam-v1706

1.4 注意事项

- 对于 FastCAE 集成开发用户，建议开展本案例学习之前，要有一定 C++基础和 OpenFoam 使用经验。
- 应尽量避免在使用 FastCAE 过程中代码存放位置及文件存放位置中包括中文路径。
- 参考 4.1 把字典文件存放到指定文件夹
- 本案例使用的几何文件取自 \$FOAM_TUTORIALS/resources/geometry/motorBike.obj.gz，案例中没有做导入操作，在 AllRun.sh 脚本中会复制到工作目录下用于网格划分。

2. 软件实现效果

本案例最终形成了基于 OpenFoam，使用 simpleFoam 求解器计算摩托车和骑手周围的稳态气流的软件系统，该系统仿真计算由 OpenFoam 内核实现，后处理功能与界面参数设置通过 FastCAE 的可视化定制功能实现，OpenFoam 字典参数设置通过 OpenFoam 适配插件实现。

在完成软件的开发工作后，用户可以运行软件进行数值计算，我们可以看到，在主界面左侧为控制面板，我们的工程树设置，关联网格，设置边界条件等都可以在工程树上进行操作。右侧主体部分为展示窗体区域，前处理窗口，后处理三维渲染窗口都在这个区域进行显示。在下面相邻的两个区域分别为控制台和进程窗口，控制台用于显示用户的操作流程，进程窗体则主要对求解状态进行显示及控制。完成的软件整体效果如下图所示。

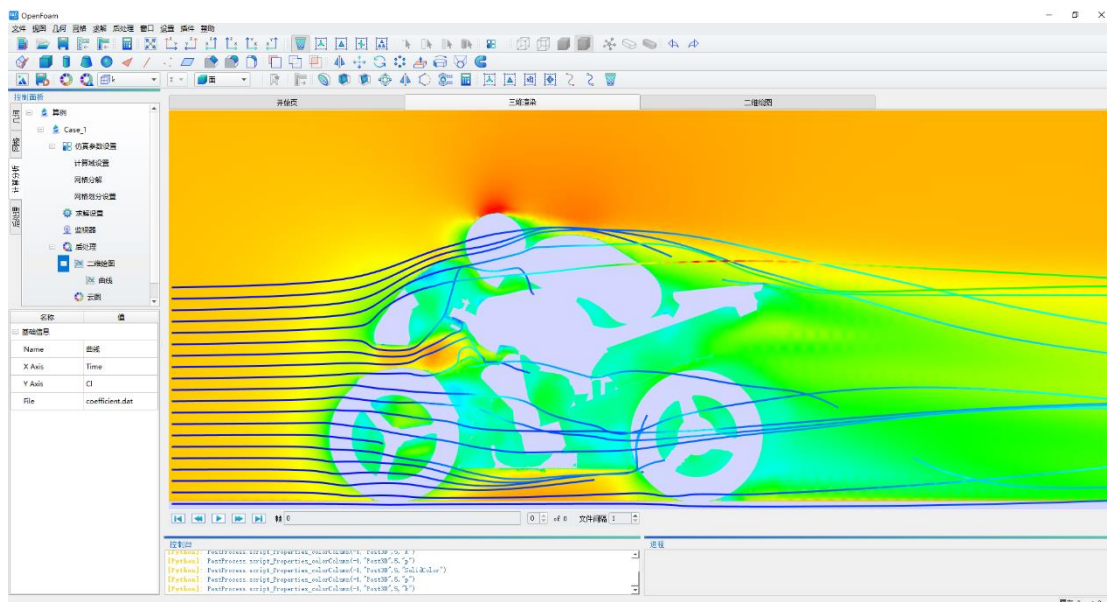


图 1 OpenFoam 案例集成软件效果

3. 原理说明

本案例的实现过程是通过 **FastCAE** 平台实现软件界面定制，并通过插件生成 **OpenFoam** 运行所需的参数文件，后台驱动 **OpenFoam** 软件进行科学计算并对计算结果进行展示。

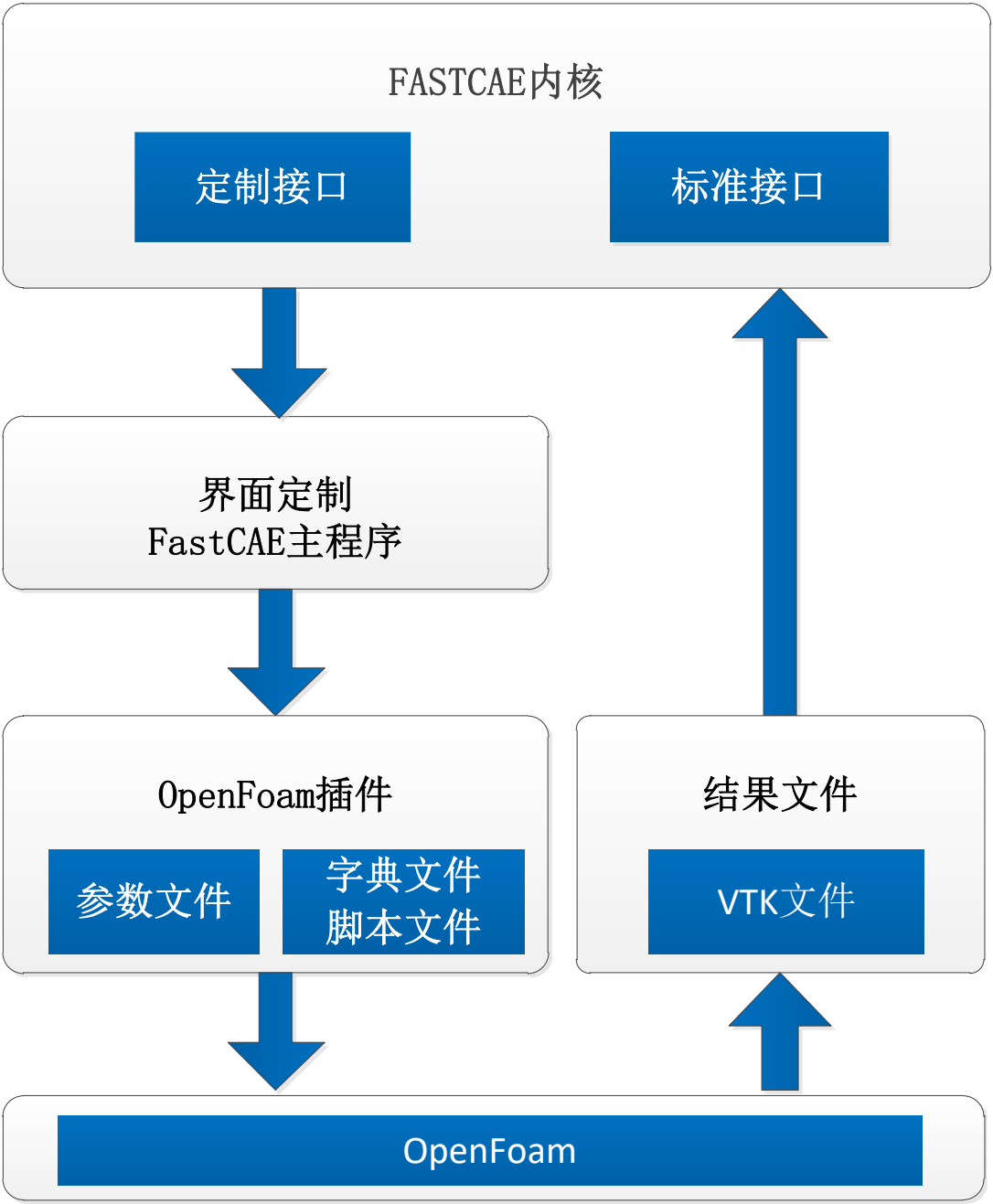


图 2 案例原理图

3.1OpenFoam 部分

3.1.1OpenFoam 常用指令

表 1 OpenFoam 指令

blockMesh	-dict	计算域网格划分
-----------	-------	---------

surfaceFeatureExtract	-dict	提取特征边
snappyHexMesh	-dict -overwrite	六面体切割器
decomposePar	-dict	网格自动分割
simpleFoam	-dict	SIMPLE 算法稳态求解
mpirun	-np -parallel	并行计算
reconstructPar		网格重组

3.1.2 OpenFoam 脚本文件说明

本案例中，我们采用脚本文件把相关指令放到脚本中执行计算

AllRun.sh

```
#!/bin/sh

cd ${0%/*} || exit 1    # Run from this directory

# Source tutorial run functions
. $WM_PROJECT_DIR/bin/tools/RunFunctions

#clean log file
./Allclean

# Alternative decomposeParDict name:
decompDict="-decomposeParDict system/decomposeParDict.6"

## Standard decomposeParDict name:

# unset decompDict
```

```

# copy motorbike surface from resources directory

\cp    $FOAM_TUTORIALS/resources/geometry/motorBike.obj.gz
constant/triSurface/

runApplication surfaceFeatureExtract


runApplication blockMesh


runApplication decomposePar $decompDict


# Using distributedTriSurfaceMesh?
if      foamDictionary      -entry      geometry      -value
system/snappyHexMeshDict | \
    grep -q distributedTriSurfaceMesh
then
    runParallel $decompDict surfaceRedistributePar motorBike.obj
independent
fi


runParallel $decompDict snappyHexMesh -overwrite


#- For non-parallel running: - set the initial fields

```

```

# restore0Dir

#- For parallel running: set the initial fields

restore0Dir -processor

runParallel $decompDict patchSummary

runParallel $decompDict potentialFoam

runParallel $decompDict $(getApplication)

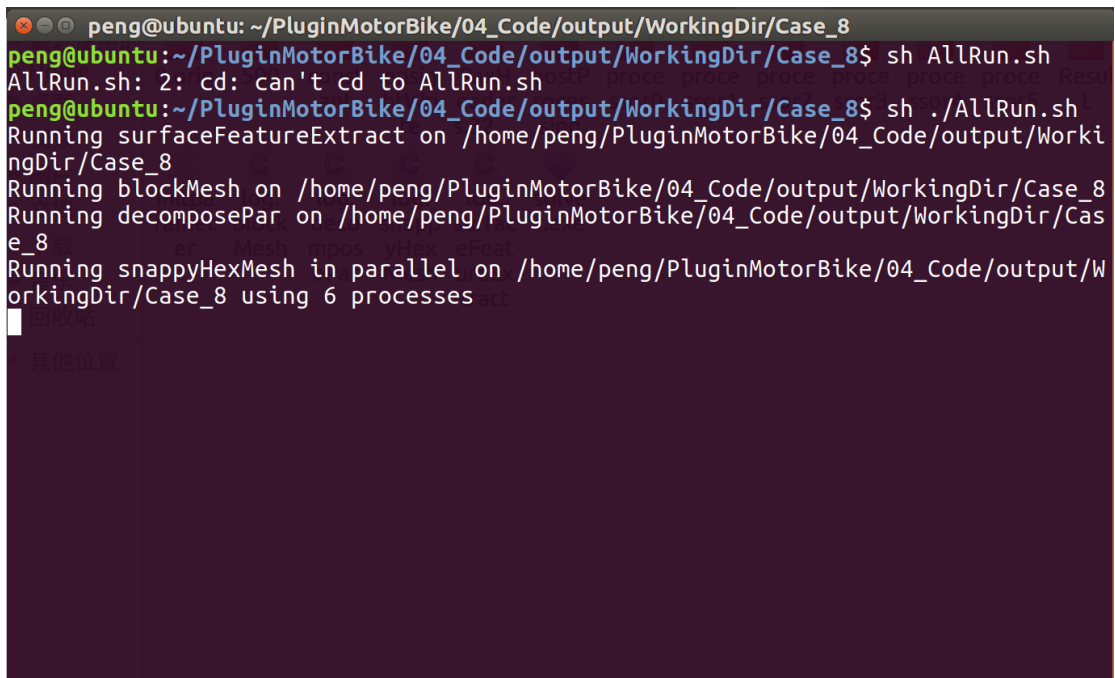
runApplication reconstructParMesh -constant

runApplication reconstructPar -latestTime

#-----

```

图 3 脚本文件



```

peng@ubuntu: ~/PluginMotorBike/04_Code/output/WorkingDir/Case_8
peng@ubuntu:~/PluginMotorBike/04_Code/output/WorkingDir/Case_8$ sh AllRun.sh
AllRun.sh: 2: cd: can't cd to AllRun.sh
peng@ubuntu:~/PluginMotorBike/04_Code/output/WorkingDir/Case_8$ sh ./AllRun.sh
Running surfaceFeatureExtract on /home/peng/PluginMotorBike/04_Code/output/WorkingDir/Case_8
Running blockMesh on /home/peng/PluginMotorBike/04_Code/output/WorkingDir/Case_8
Running decomposePar on /home/peng/PluginMotorBike/04_Code/output/WorkingDir/Case_8
Running snappyHexMesh in parallel on /home/peng/PluginMotorBike/04_Code/output/WorkingDir/Case_8 using 6 processes

```

图 4 命令行下运行脚本

如上图，sh 脚本会执行 OpenFoam 相关指令进行运算

3.2FastCAE 部分

根据 3.0 原理图，我们可以分析得出 **FastCAE** 需要调用脚本文件让 **OpenFoam** 在后台运行，即我们在 **FastCAE** 部分需要在求解过程中完成 **initparameter** 文件的生成、字典文件及脚本文件复制到算例目录下。

其中，**FastCAE** 定制化插件部分及**软件框架**将提供软件参数的设置、网格的导入及展示、后处理结果展示功能。这部分功能不需要用户重新开发，只需要进行配置就能够使用。

对于用户需要新增加参数的情况，需要在 **FastCAE** 框架中通过定制化插件配置好新增加的参数，同时在字典文件中进行参数的关联，可以参考 4.1 和 4.3 部分

3.3算例变量

表 2 变量说明

变量类型	变量名称	类型	默认值
计算域设置	顶点坐标	表格型	(8 行 3 列) -5,-4,0 15,-4,0 15,4,0 -5,4,0

			-5,-5,8 15,-4,8 15,4,8 -5,4,8
	各方向网格数量	表格型	(1 行 3 列) 20,8,8
网格分解	分解域的总数	整形	6
	各方向分解数量	表格形	(1 行 3 列) 6,1,1
	分解方法	字符串枚举	hierarchical ptscotch
网格划分设置	是否切割网格	Bool	true
	是否进行网格贴合	Bool	ture
	添加网格边界层	Bool	true
	最小表面细化等级	整形	5
	最大表面细化等级	整形	6
	边界层添加迭代数	整形	50
求解设置	求解器名称	字符串	simpleFoam

	文件生成间隔	整形	100
	开始步	整形	0
	结束步	整形	500

4. 操作/开发步骤

本案例开发包括四个步骤：字典操作、脚本文件生成、软件界面定制、插件开发。

- 字典操作：从 OpenFoam 指定路径下获取字典文件添加参数
- 脚本文件生成：生成 AllRun.sh 脚本文件
- 软件界面定制：实现界面相关参数设置
- 插件开发：实现求解输入文件生成及求解后结果文件复制

4.1 字典操作

通过分析 OpenFoam 命令得知，OpenFoam 执行指令需要通过读取字典文件完成，字典文件引用 `initparameter` 参数文件实现参数的设置，字典文件需要用户根据算例需求提前设置好，放到指定目录下，字典文件如下图所示：

```

1 |/*-----*- C++ -*/
2 |
3 |=====
4 | \ \ \ \ \ F i e l d      | OpenFOAM: The Open Source CFD Toolbox
5 | \ \ \ \ \ O peration    | Version: plus
6 | \ \ \ \ \ A nd          | Web: www.OpenFOAM.com
7 | \ \ \ \ \ M anipulation  |
8 |-----*/
9 |FoamFile
10 |{
11 |    version      2.0;
12 |    format        ascii;
13 |    class         dictionary;
14 |    object        decomposeParDict;
15 |}
16 |#include "../initparameter"
17 |// *****
18 |numberOfSubdomains $core;
19 |
20 |method            hierarchical;
21 |// method         ptscotch;
22 |
23 |simpleCoeffs
24 |{
25 |    n              ($nx $ny $nz);
26 |    delta          0.001;
27 |}
28 |
29 |hierarchicalCoeffs
30 |{
31 |    n              (3 2 1);
32 |    delta          0.001;
33 |    order          xyz;
34 |}
35 |
36 |manualCoeffs
37 |{
38 |    dataFile       "cellDecomposition";
39 |}
40 |
41 |
42 |// *****

```

关联输入参数

对应initparameter文件中的参数

图 5 字典关联参数文件

Initparameter 参数文件由 FastCAE 调用求解器前生成，具体内容如下图所示：

```
initparameter
1 starttime.....0;
2 endtime.....500;
3 application.....simpleFoam;
4 writeInterval.....100;
5 xgridnumber.....20;
6 ygridnumber.....8;
7 zgridnumber.....8;
8 x1.....-5;
9 y1.....-4;
10 z1.....0;
11 x2.....15;
12 y2.....-4;
13 z2.....0;
14 x3.....15;
15 y3.....4;
16 z3.....0;
17 x4.....-5;
18 y4.....4;
19 z4.....0;
20 x5.....-5;
21 y5.....-5;
22 z5.....8;
23 x6.....15;
24 y6.....-4;
25 z6.....8;
26 x7.....15;
27 y7.....4;
28 z7.....8;
29 x8.....-5;
30 y8.....4;
31 z8.....8;
32 nx.....6;
33 ny.....1;
34 nz.....1;
35 core.....6;
36 decomposemethod.....hierarchical;
37 castellatedMesh.....true;
38 snap.....true;
39 addLayers.....true;
40 minlevel.....5;
41 maxlevel.....6;
42 nLayerIter.....50;
```

图6 initparameter 文件

4.1.1 复制字典文件

- 在 FastCAE 2.0 应用所在路径的上级目录下创建 solver/dictionary 文件夹，此算例中 dictionary 全路径为 PluginMotorBike/04_Code/Output/solver/dictionary。

FastCAE2.0 的全路径为 PluginMotorBike/04_Code/Output/bin

- 将 \$FOAM_TUTORIALS \incompressible\simpleFoam\motorBike 文件夹中所有文件复制到 dictionary 文件夹下（也可以使用我们提供配置好的字典文件），Linux 下命令如下：

```
cp -r $FOAM_TUTORIALS\incompressible\simpleFoam\motorBike .
```

- linux 下需要将 Allclean, AllRun.sh 赋予运行权限，如下图所示：

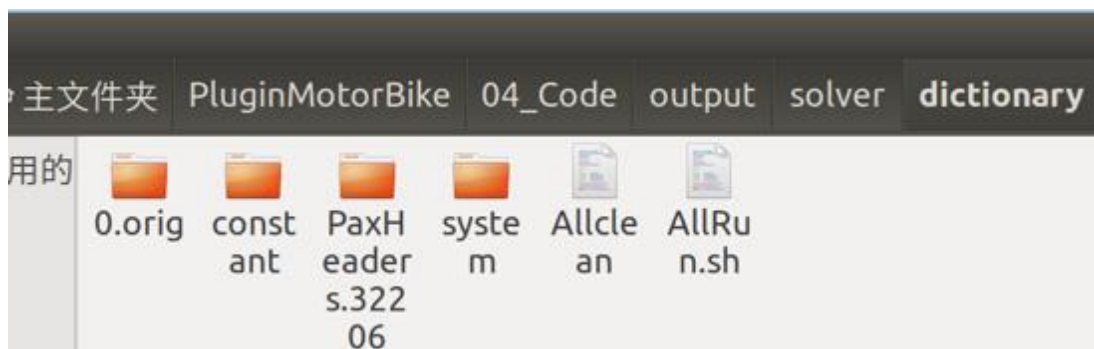


图 7 字典文件路径

AllRun.sh 驱动求解计算的脚本文件

Allclean 清除算例日志文件

Linux 更改权限命令如下：

```
chmod 777 AllRun.sh
```

```
chmod 777 Allclean
```

4.1.2 字典文件关联参数

OpenFoam 可通过修改字典文件实现相关指令参数的修改，在字典文件中包含 `initparameter` 参数文件，用于参数的修改，可用文本编辑工具（`notepad++`）修改字典文件，本例中字典文件路径 `PluginMotorBike/04_Code/output/solver/dictionary`（用的是我们提供的配置好的字典文件可以不做此操作）

字典中的参数采用 `$parameter` 格式指定，`parameter` 为参数名称，此名称要与 `initparameter` 文件中的变量名称一致。本案例中需要修改 `blockMeshDict`、`decomposeParDict.6`、`controlDict`、`snappyHexMeshDict` 字典文件，修改过程如下：

- 修改 `blockMeshDict` 字典文件

文件所在路径 `/PluginMotorBike/04_Code/output/solver/dictionary/system/blockMeshDict`

```
1  /*-----*- C++ -*-----*\
2  |=====| |OpenFOAM: The Open Source CFD Toolbox|
3  |\\| |Operation| |Version: plus|
4  |\\| |And| |Web: www.OpenFOAM.com|
5  |\\| |Manipulation|
6  |-----*\
7  FoamFile
8  {
9  ....version.....2.0;
10 ....format.....ascii;
11 ....class.....dictionary;
12 ....object.....blockMeshDict;
13 }
14 #include "../initparameter"
15 // .....//
16
17 convertToMeters 1;
18
19 vertices
20 (
21 .... ($x1 $y1 $z1)
22 .... ($x2 $y2 $z2)
23 .... ($x3 $y3 $z3)
24 .... ($x4 $y4 $z4)
25 .... ($x5 $y5 $z5)
26 .... ($x6 $y6 $z6)
27 .... ($x7 $y7 $z7)
28 .... ($x8 $y8 $z8)
29 );
30
31 blocks
32 (
33 .... hex (0 1 2 3 4 5 6 7) ($xgridnumber $ygridnumber $zgridnumber) simpleGrading (1 1 1)
34 );
35
36 edges
37 (
38 );
39
40 boundary
41 (
42 .... frontAndBack
43 .... {
44 .... type patch;
45 .... faces
46
```

图 8 blockMeshDict 字典

对于“vertices”，现在有 8 个顶点，则有 8 行，分别为：

(\$x1 \$y1 \$z1)

(\$x2 \$y2 \$z2)

(\$x3 \$y3 \$z3)

(\$x4 \$y4 \$z4)

(\$x5 \$y5 \$z5)

(\$x6 \$y6 \$z6)

(\$x7 \$y7 \$z7)

(\$x8 \$y8 \$z8)

如有新增顶点，则可以按顺序增加顶点坐标，如\$x9 \$y9 \$z9 等。

对于 blocks，\$xgridnumber \$ygridnumber \$zgridnumber 用于设置各方向网格数量，如下图所示：

```
blocks
(
  ...hex (0 1 2 3 4 5 6 7) ($xgridnumber $ygridnumber $zgridnumber) simpleGrading (1 1 1)
);
```

图 9 各方向上网格数量

界面上计算域的设置，如下图所示：

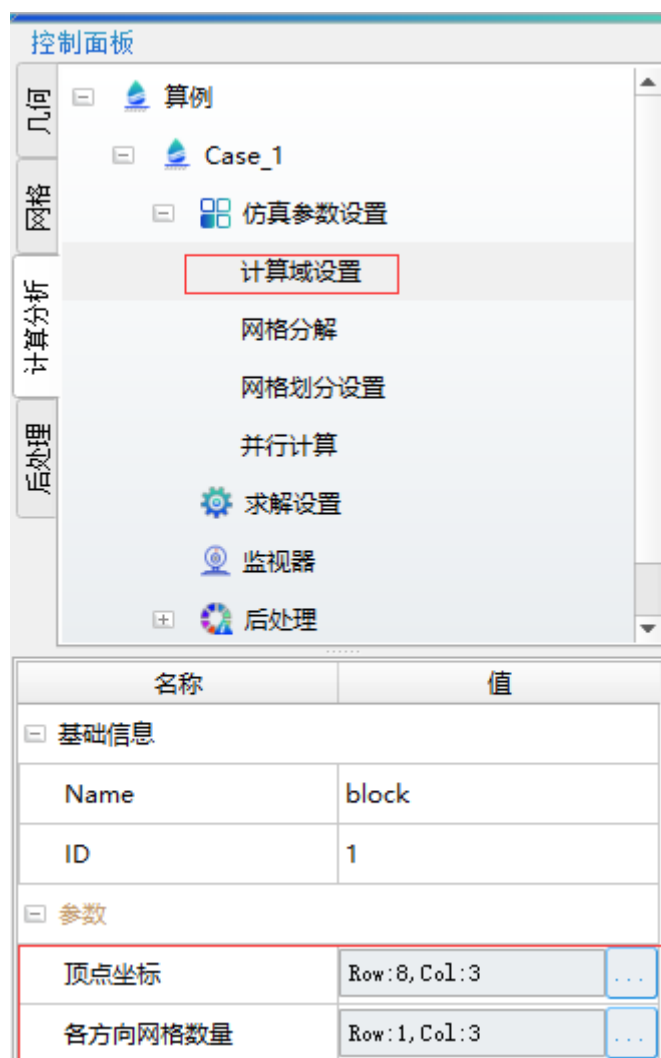


图 10 计算域设置

- 修改 decomposeParDict.6 字典文件

文件所在路径: System/decomposeParDict.6

```
1 /*-----*-- C++ --*-----*/
2 |=====|
3 |\\...../ Field.....| OpenFOAM: The Open Source CFD
4 |\\...../ Operation.....| Version: plus
5 |\\...../ And.....| Web: www.OpenFOAM.com
6 |\\...../ Manipulation.....|
7 /*-----*-- C++ --*-----*/
8 FoamFile
9 {
10 ..version.....2.0;
11 ..format.....ascii;
12 ..class.....dictionary;
13 ..object.....decomposeParDict;
14 }
15 #include "../initparameter"
16 //*****
17
18 numberOfSubdomains $core;
19
20 method.....hierarchical;
21 // method.....ptscotch;
22
23 simpleCoeffs
24 {
25 ..n.....($nx $ny $nz);
26 ..delta.....0.001;
27 }
28
29 hierarchicalCoeffs
30 {
31 ..n.....(3 2 1);
32 ..delta.....0.001;
33 ..order.....xyz;
34 }
35
36 manualCoeffs
37 {
38 ..dataFile....."cellDecomposition";
39 }
40
```

图 11 decomposeParDict 字典

\$nx,\$ny,\$nz 用于关联各方向分解数量,如下图所示:

```
simpleCoeffs
{
..n.....($nx $ny $nz);
..delta.....0.001;
}
```

图 12 各方向分解数量

\$score 用于关联计算核数,如下图所示:

```
numberOfSubdomains.$score;
```

图 13 计算核数设置

注: nx,ny,nz 相乘之和要与\$score 参数相等, 否则网格划分过程中会出错。

界面上网格分解参数, 如下图所示:



图 14 网格分解参数

3、修改 controlDict 字典文件

所 在 路 径 /PluginMotorBike/04_Code/output/solver/

dictionary/system/controlDict

```

controlDict
1  /*-----*--C++-*-----
2  |=====|
3  |\\.Field|OpenFOAM:The Open Sc
4  |\\.Operation|Version: plus
5  |\\.And|Web: www.OpenFOA
6  |\\.Manipulation|
7  \*-----
8  FoamFile
9  {
10  ....version....2.0;
11  ....format....ascii;
12  ....class....dictionary;
13  ....object....controlDict;
14  }
15  //*****
16  #include "../initparameter"
17  application....$application;
18
19  startFrom....latestTime;
20
21  startTime....$starttime;
22
23  stopAt....endTime;
24
25  endTime....$endtime;
26
27  deltaT....1;
28
29  writeControl....timeStep;
30
31  writeInterval....$writeInterval;
32
33  purgeWrite....0;
34
35  writeFormat....binary;
36
37  writePrecision....6;
38
39  writeCompression off;
40
41  timeFormat....general;
42
43  timePrecision....6;
44
45  runtimeModifiable true;
46

```

图 15 controlDict 字典

用于关联求解设置参数，如下图所示：

求解器名称	simpleFoam
文件生成间隔	100
开始步	0
结束步	500
启用并行	☐

图 16 界面求解器参数

\$application 关联求解器名称

\$writeInterval 关联文件生成间隔

\$starttime 关联开始时间

\$endtime 用于关联结束时间

4、修改 snappyHexMeshDict 字典文件

所 在 路 径 /PluginMotorBike/04_Code/output/solver/
dictionary/system/snappyHexMeshDict

```

1  /*-----*-----C++-----*-----*\
2  |=====|-----|
3  |\\|Field|OpenFOAM: The Open Source CFD Toolbox|
4  |\\|Operation|Version: plus|
5  |\\|And|Web: www.OpenFOAM.com|
6  |\\|Manipulation|
7  \*-----*-----*\
8  FoamFile
9  {
10     ....version....2.0;
11     ....format....ascii;
12     ....class....dictionary;
13     ....object....snappyHexMeshDict;
14 }
15 //*****//
16 #include "../initparameter"
17 //Which of the steps to run
18 castellatedMesh $castellatedMesh;
19 snap .....$snap;
20 addLayers .....$addLayers;
21
22
23 //Geometry Definition of all surfaces. All surfaces are of class
24 //searchableSurface.
25 //Surfaces are used
26 //--to specify refinement for any mesh cell intersecting it
27 //--to specify refinement for any mesh cell inside/outside/near
28 //--to 'snap' the mesh boundary to the surface
29 geometry
30 {
31     ....motorBike.obj
32     ....{
33         ....type triSurfaceMesh;
34         ....name motorBike;
35     }
36
37     ....refinementBox
38     ....{
39         ....type searchableBox;
40         ....min (-1.0 -0.7 0.0);
41         ....max ( 8.0  0.7 2.5);
42     }
43 };

```

图 17 snappyHexMeshDict 字典

```

....refinementSurfaces
....{
....    motorBike
....    {
....        ....//Surface-wise min and max refinement level
....        ....level ($minlevel $maxlevel);

....        ....//Optional specification of patch type (default is wall). No
....        ....//constraint types (cyclic, symmetry) etc. are allowed.
....        ....patchInfo
....        {
....            ....type wall;
....            ....inGroups (motorBikeGroup);
....        }
....    }
....}

```

图 18 最大最小值参数

```

275 .....maxThicknessToMedialRatio:0.3;
276
277 .....//Angle used to pick up medial axis points
278 .....//Note: changed(corrected) w.r.t.17x!.90 degrees corresponds to.130.in.17
279 .....minMedianAxisAngle:90;
280
281
282 .....//Create buffer region for new layer terminations
283 .....nBufferCellsNoExtrude:0;
284
285
286 .....//Overall max number of layer addition iterations. The mesher will exit
287 .....//if it reaches this number of iterations; possibly with an illegal
288 .....//mesh.
289 .....nLayerIter:$nLayerIter;
290 }
291

```

图 19 边界层参数

界面求解设置参数，如下图所示：

名称	值
基础信息	
Name	SnappyHexMeshSetti...
ID	1
参数	
是否切割网格	☒
是否进行网格贴合	☒
添加网格边界层	☒
最小表面细化等级	5
最大表面细化等级	6
边界层添加迭代数	50

图 20 界面 SnappyHexMesh 参数

\$ castellatedMesh 关联是否切割网格

\$snap 是否进行网格贴合

\$ addLayers 添加网格边界层

\$minlevel 最小表面细化等级

\$ maxlevel 最大表面细化等级

\$ nLayerIter 边界层添加迭代数

4.2 脚本文件

案例中用到的脚本文件为 **Allclean** 和 **AllRun.sh** 文件, **AllRun.sh** 用于启动求解计算, **Allclean** 为清空历史日志文件及网格文件等, 要在求解计算前先调用 **Allclean** 脚本。

AllRun.sh 和 **Allclean** 文件可通过下载获取, 也可以在此基础上自行扩展。

本 案 例 中 脚 本 文 件 存 放 路 径 为
PluginMotorBick/04_Code/Output/solver/dictionaray

AllRun.sh 脚本文件内容如下所示:

```
#!/bin/sh

cd ${0%/*} || exit 1    # Run from this directory

# Source tutorial run functions

. $WM_PROJECT_DIR/bin/tools/RunFunctions

#clean log file

./Allclean

# Alternative decomposeParDict name:

decompDict="-decomposeParDict system/decomposeParDict.6"

## Standard decomposeParDict name:

# unset decompDict


# copy motorbike surface from resources directory

\cp    $FOAM_TUTORIALS/resources/geometry/motorBike.obj.gz
```

```
constant/triSurface/
```

```
runApplication surfaceFeatureExtract
```

```
runApplication blockMesh
```

```
runApplication decomposePar $decompDict
```

```
# Using distributedTriSurfaceMesh?
```

```
if      foamDictionary      -entry      geometry      -value
```

```
system/snappyHexMeshDict | \
```

```
    grep -q distributedTriSurfaceMesh
```

```
then
```

```
    runParallel $decompDict surfaceRedistributePar motorBike.obj
```

```
independent
```

```
fi
```

```
runParallel $decompDict snappyHexMesh -overwrite
```

```
#- For non-parallel running: - set the initial fields
```

```
# restore0Dir
```

```
#- For parallel running: set the initial fields
```

```
restore0Dir -processor
```

```

runParallel $decompDict patchSummary
runParallel $decompDict potentialFoam
runParallel $decompDict $(getApplication)
runApplication reconstructParMesh -constant
runApplication reconstructPar -latestTime

#-----

```

图 21 脚本文件说明

主要脚本文件描述如下：

- decompDict="-decomposeParDict
system/decomposeParDict.6"

用于设置网格分割需要的字典文件参数

- \cp \$FOAM_TUTORIALS/resources/geometry/motorBike.obj.gz
constant/triSurface/

从 OpenFoam 安装目录下复制网格文件到字典目录
constant/triSurface/

提取特征边

- runApplication surfaceFeatureExtract

划分计算域网格

- runApplication blockMesh

网格自动分割

- runApplication decomposePar

六面体网格切割

- runParallel \$decompDict snappyHexMesh –overwrite
- 调用求解器求解计算
- runParallel \$decompDict \$(getApplication)

网格重组

- runApplication reconstructPar –latestTime

4.3 软件界面定制

使用 FASTCAE 定制插件完成，OpenFoam 案例界面定制，定制内容如下表所示：

表 3 软件定制内容

编号	定制步骤说明
1	加载定制化插件
2	添加软件基本信息
3	网格相关设置
4	网格导出设置
5	算例设置
6	添加计算域设置
7	网格分解设置
8	网格划分设置
9	添加求解设置
10	后处理二维曲线设置

4.3.1 加载定制化插件

点击【插件】-【插件管理】，启动 FastCAE 2.0 版本软件加载 PluginCustomizer.dll 插件，开始进行软件常规项定制。

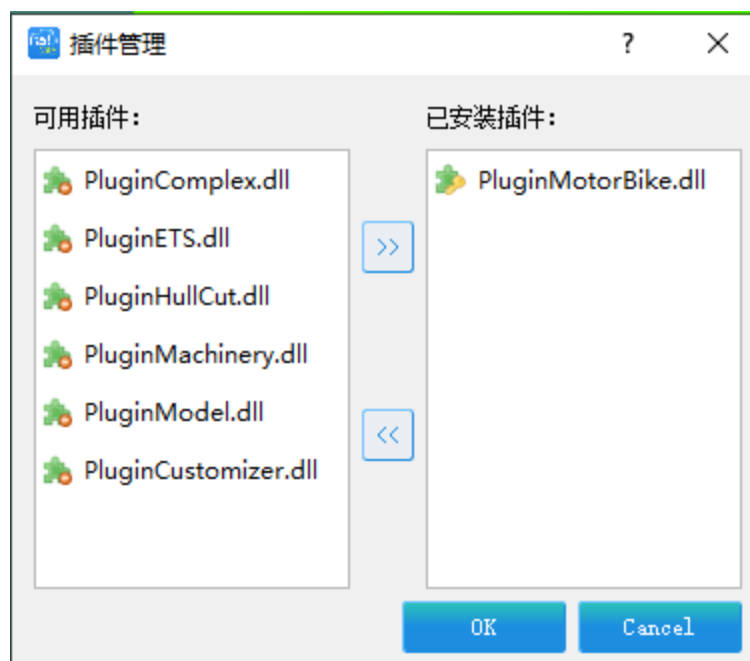


图 22 插件管理

在可用插件列表中选 **PluginCustomizer.dll** 插件，点击  按钮。

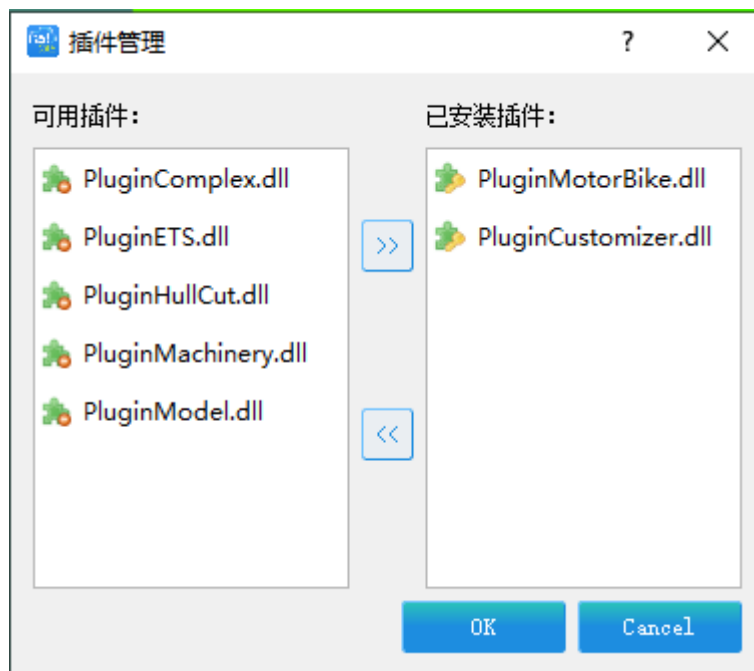


图 23 加载插件

点击  按钮，这样 PluginCustomizer 插件将会被 FastCAE 软件装载。软件菜单栏上将会出现定制项。

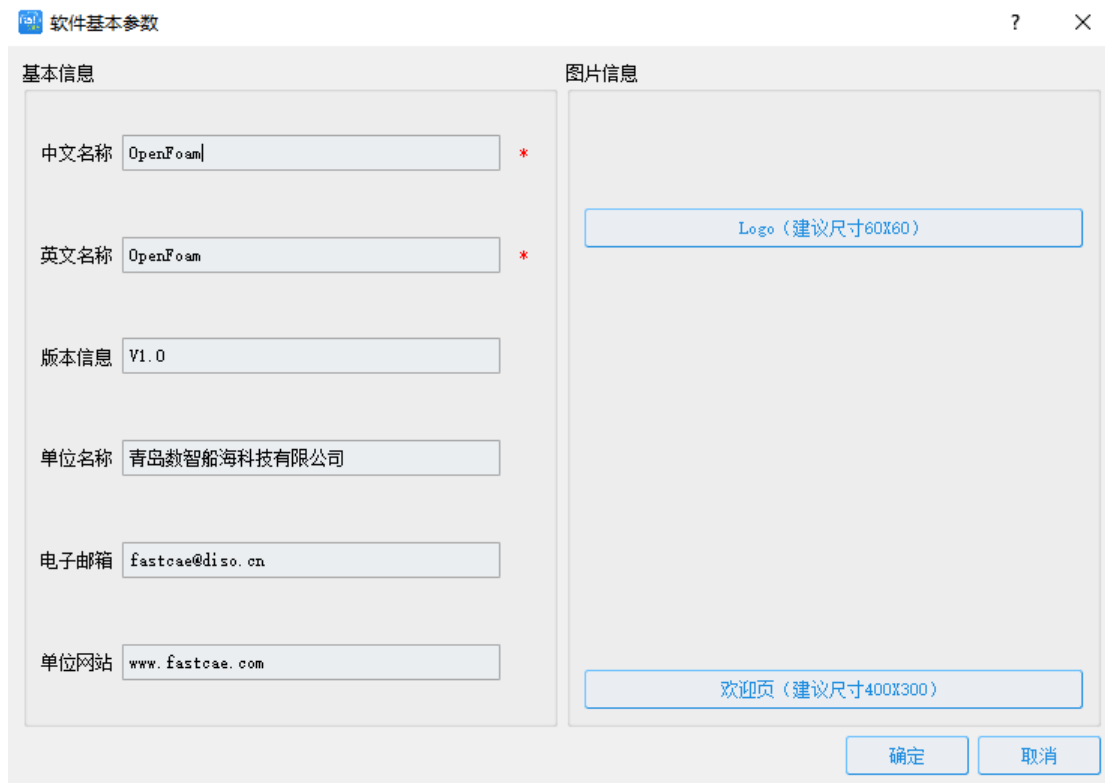


点击【定制】-【开始定制】，软件标题栏上名称后面会有“---定制中”字样，此时我们处于定制模式。



4.3.2 添加基本信息

点击【帮助】-【关于】，软件将弹出软件基本信息设置界面，这里面我们按照下图输入我们想要填入的软件基本信息。然后点击  按钮，完成软件基本信息的录入。



The image shows a 'Software Basic Parameters' dialog box with two main sections: 'Basic Information' and 'Image Information'. The 'Basic Information' section contains input fields for 'Chinese Name' (OpenFoam), 'English Name' (OpenFoam), 'Version Information' (V1.0), 'Unit Name' (青岛数智船海科技有限公司), 'Email' (fastcae@diso.cn), and 'Unit Website' (www.fastcae.com). The 'Image Information' section contains two image placeholders: 'Logo (recommended size 60x60)' and 'Welcome Page (recommended size 400x300)'. At the bottom right are 'Confirm' and 'Cancel' buttons.

Field	Value
中文名称	OpenFoam
英文名称	OpenFoam
版本信息	V1.0
单位名称	青岛数智船海科技有限公司
电子邮箱	fastcae@diso.cn
单位网站	www.fastcae.com

图 24 基本参数

4.3.3 网格相关设置

由于我们 OpenFoam 案例不需要网格剖分、网格质量检查和创建组件功能，为保证最终软件最简化，我们定制去掉这部分功能。点击【网格】，在菜单中将【面网格划分】、【体网格划分】、【网格质量检查】、【创建网格组件】菜单前面的勾去掉。

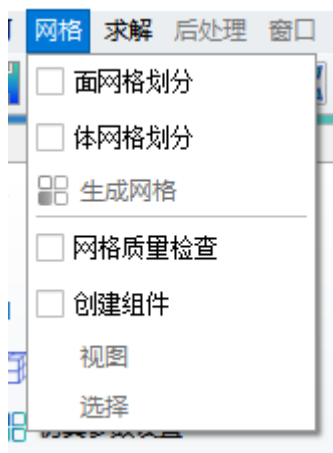


图 25 网格相关

我们的 OpenFoam 案例从 OpenFoam 安装目录下读取网格文件，所以我们去掉网格文件导入

4.3.4 网格导出设置

我们的案例不需要导出网格功能，为保证最终软件最简化，我们定制将该部分功能去掉。点击【文件】-【导出网格】，软件将弹出到处网格功能定制界面，我们将全选前面的勾选的勾去掉，确保勾选项处于反选状态，点击  按键，完成导出网格功能的定制，如下图所示：

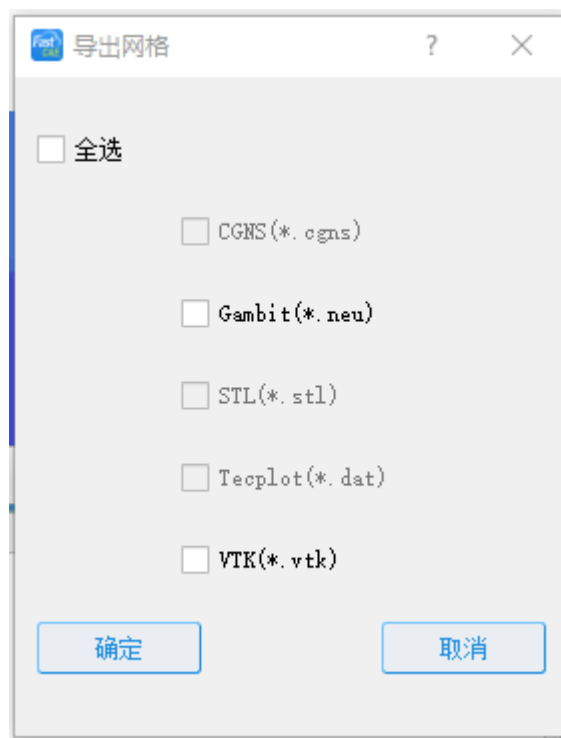


图 26 网格导出设置

4.3.5 算例设置

下面我们进行模型树的定制，将控制面板切换到计算分析分页。



图 27 算例节点

鼠标左键选中算例节点，点击鼠标右键，点击弹出右键菜单的【添加算例】菜单项。

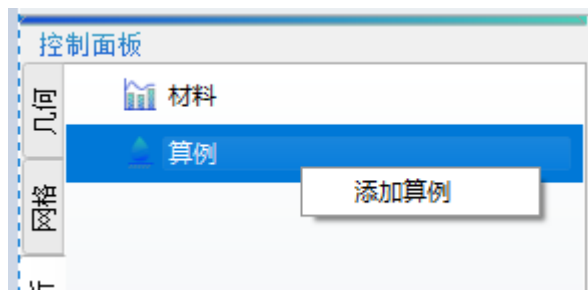
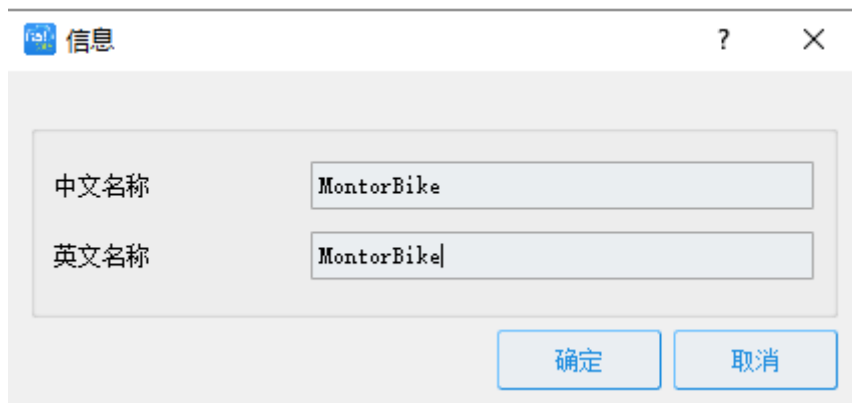


图 28 添加算例

软件将弹出添加算例对话框，我们按照下图添加入算例中文名称

和英文名称。点击  按钮完成算例名称信息的录入。



此时，【控制面板】-【算例】节点下面将会有有一个用于定制的算例树。

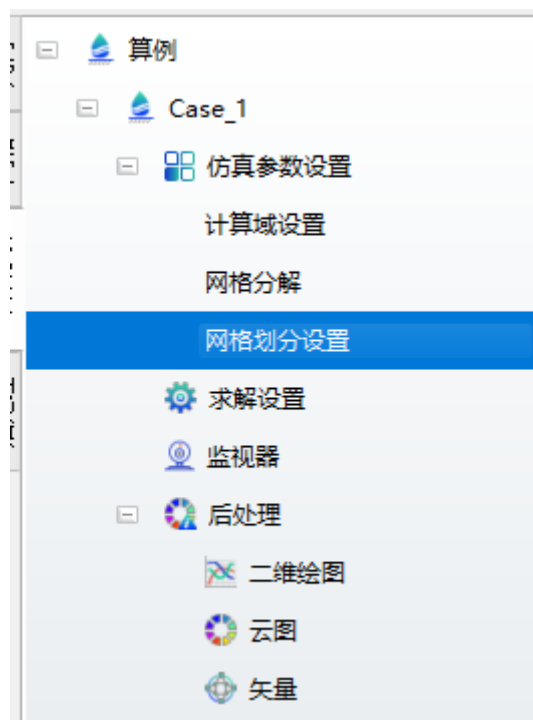


图 29 网格划分设置节点

4.3.6 添加计算域设置

鼠标左键单击算例树下的【仿真参数设置】节点，右键弹出【添加子项】，添加“计算域设置”节点。

- 1) 双击“计算域设置”节点，添加“顶点坐标”和“各方向网格数量”表格参数。

（用户配置参数名称要与文档中给出的变量名称一致，因为生成参数文件时会用到此名称去获取对应的参数值）

	x	y	z
1	-5	-4	0
2	15	-4	0
3	15	4	0
4	-5	4	0
5	-5	-5	8
6	15	-4	8
7	15	4	8
8	-5	4	8

Export Import OK Apply Cancel

图 30 项点坐标

表格类型参数

?

×

名称

各方向网格数里

行数

1

列数

3

默认值

刷新表格

x	y	z
20	8	8

确定

取消

图 31 各方向网格数量

计算域设置

?

×

基本信息

中文名称

计算域设置

英文名称

block

图标

上传图标

参数列表

添加

删除

编辑

清空全部

序号	名称	类型	默认值
1	顶点坐标	Table	8x3
2	各方向网格数量	Table	1x3

显示参数组

确定

取消

图 32 计算域设置

4.3.7 网格分解设置

鼠标左键单击算例树下的【仿真参数设置】节点，右键弹出【添加子项】，添加“网格分解设置”

双击“网格分解”节点，添加“分解域的总数”、“各方向分解数量”、“分解方法”。

整型参数

?

×

名称

分解域的总数

单位

默认值

6

最小值

-1000000

最大值

1000000

确定

取消

图 33 分解域的总数

表格类型参数

?

×

名称

各方向分解数量

行数

1

列数

3

默认值

刷新表格

x	y	z
6	1	1

确定

取消

图 34 各方向分解数量

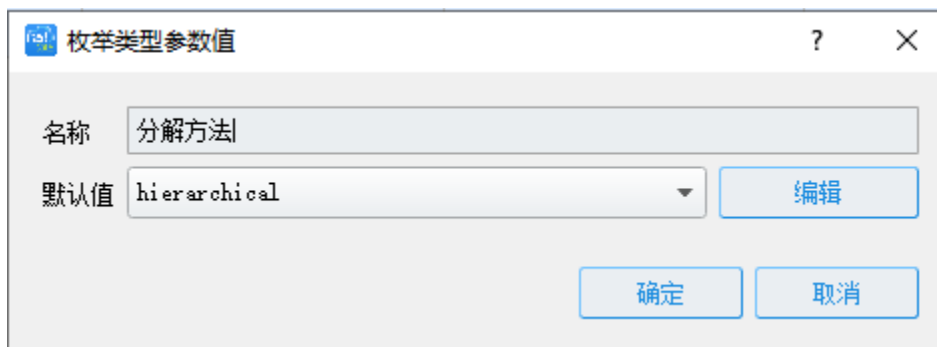


图 35 分解方法



可以添加 hierarchical 和 ptscotch 两种分解方法，默认用 hierarchical 方法。

图 36 添加分解方法枚举值

4.3.8 网格划分设置

鼠标左键单击算例树下的【仿真参数设置】节点，右键弹出【添加子项】，添加“网格划分设置”

右键点击“网格划分设置”节点，添加“是否切割网格”、“是否进

行网格贴合“、”添加网格边界层”、“最小表面细化等级”、“最大表面细化等级”、“边界层添加迭代数”

网格划分设置

基本信息

中文名称

网格划分设置

英文名称

SnappyHexMeshSetting

图标

上传图标

参数列表

添加

删除

编辑

清空全部

序号	名称	类型	默认值
1	是否切割网格	Bool	true
2	是否进行网格贴合	Bool	true
3	添加网格边界层	Bool	true
4	最小表面细化等级	Int	5
5	最大表面细化等级	Int	6
6	边界层添加迭代数	Int	50

显示参数组

确定

取消

图 37 网格划分设置

4.3.9 添加求解设置

鼠标左键双击【控制面板】-【算例】节点下的【求解设置】节点，软件将弹出求解参数设置界面,添加“求解器名称”，“文件生成间隔”，“开始步”，“结束步”，“启用并行”等参数。

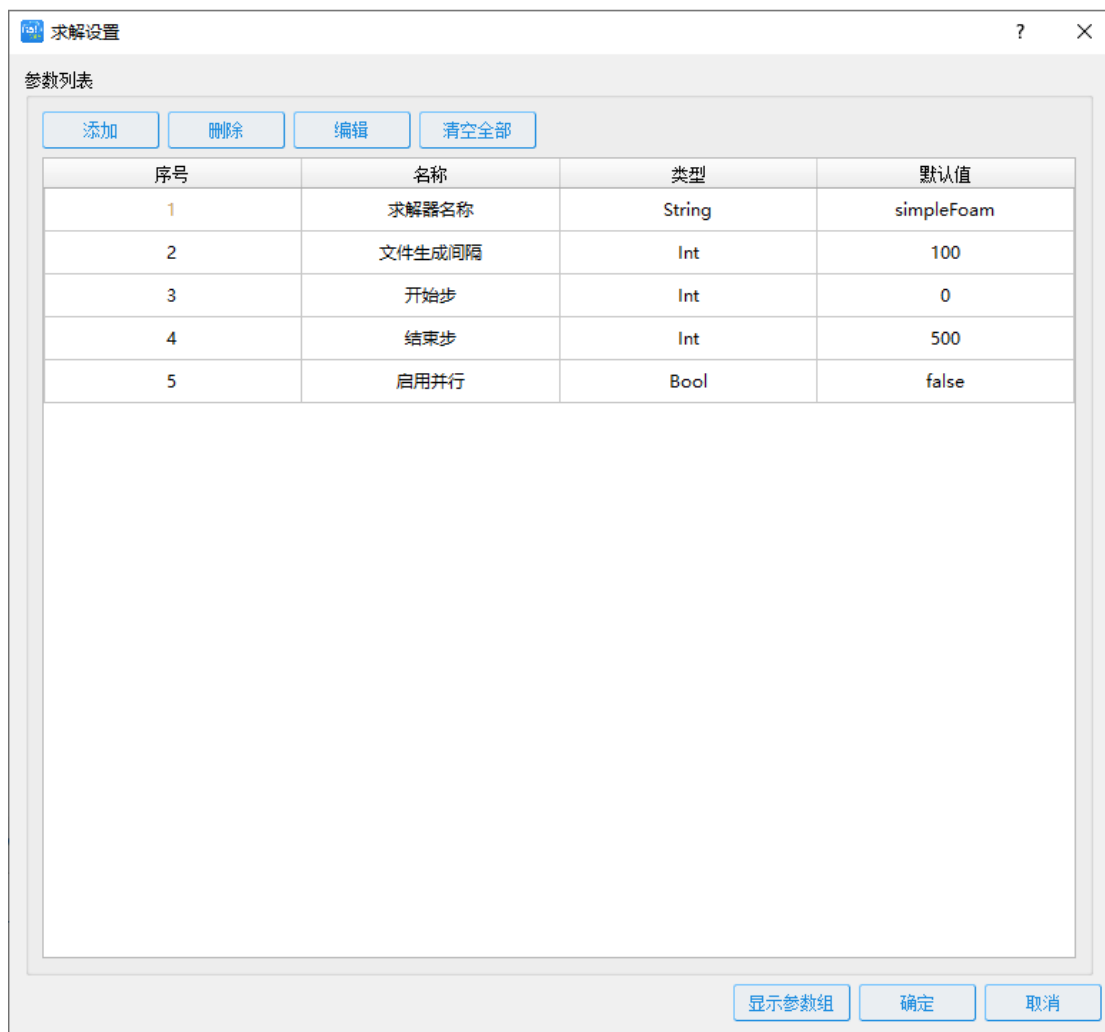


图 38 求解参数设置

4.3.10 后处理二维曲线设置

本算例求解完成后，二维文件默认保存在 `postProcessing` 目录下不符合 FastCAE 后处理文件路径读取规则，如下图所示：

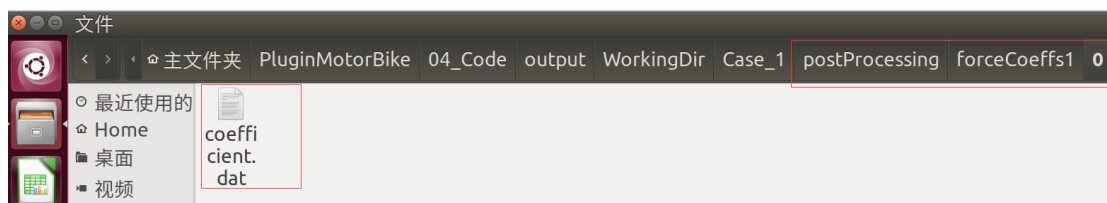


图 39 求解完成后存放路径

求解完成后，需通过转换函数将结果文件复制算例目录下的 `Result/ coefficient.dat` 文件（已在插件中完成此功能）。如下图所示：

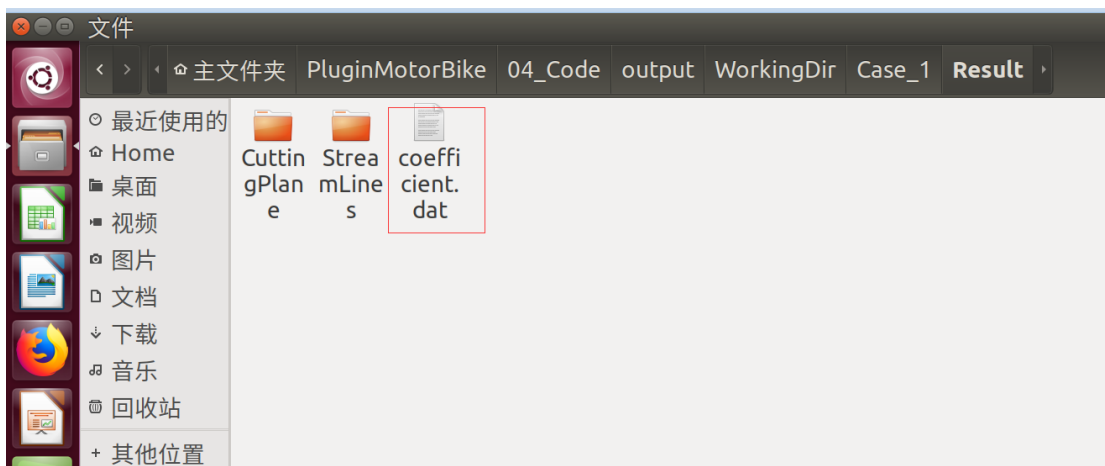


图 40 二维曲线文件转换后存放路径

在“二维后处理”节点上，按右键弹出“添加文件”对话框，如下图所示：

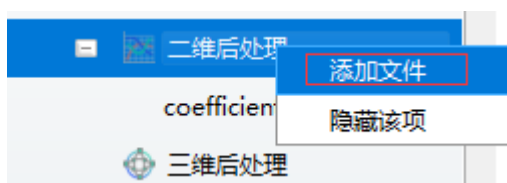


图 41 添加文件

单击“添加文件”弹出如下对话框，如下图所示：

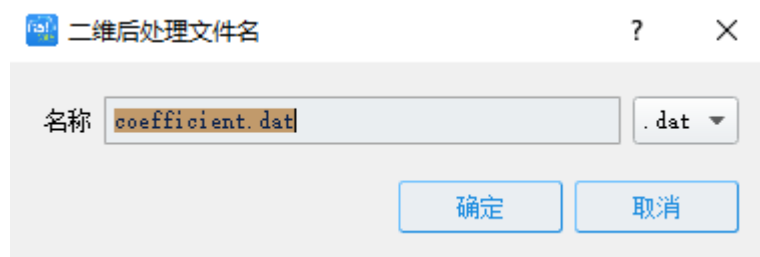


图 42 二维文件名

添加 x 轴和 y 轴变量名称，如下图所示：

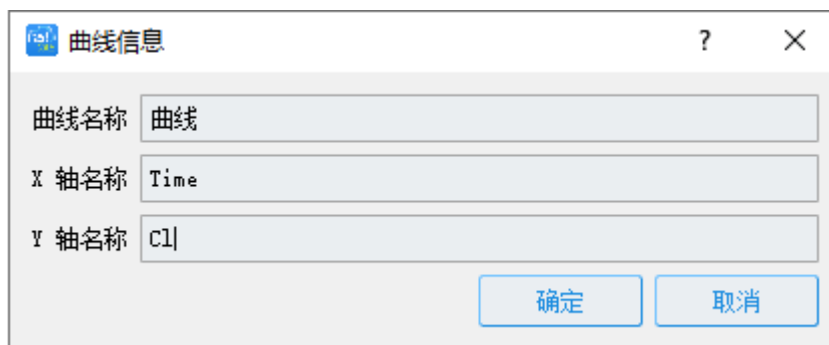


图 43 变量名称

曲线参数设置如下图所示：

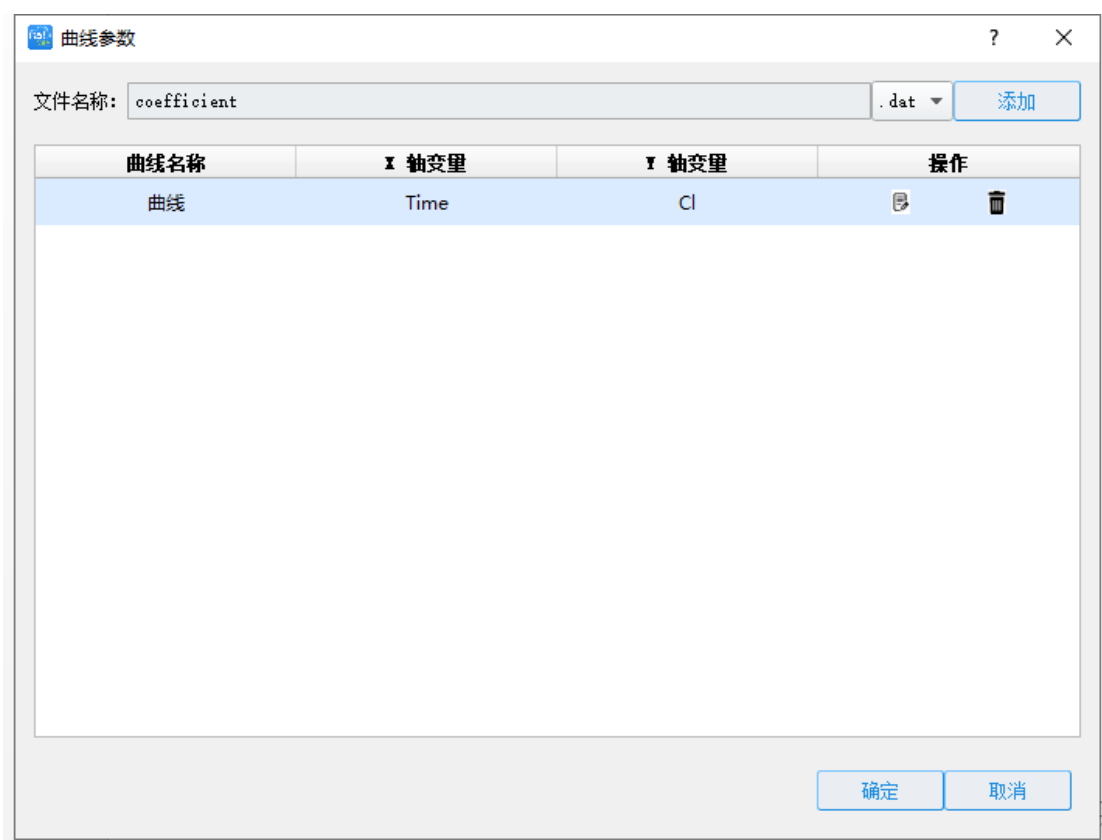


图 44 曲线参数

在“二维曲线”节点上，单击“右键”显示出对应的二维曲线，如下图所示：

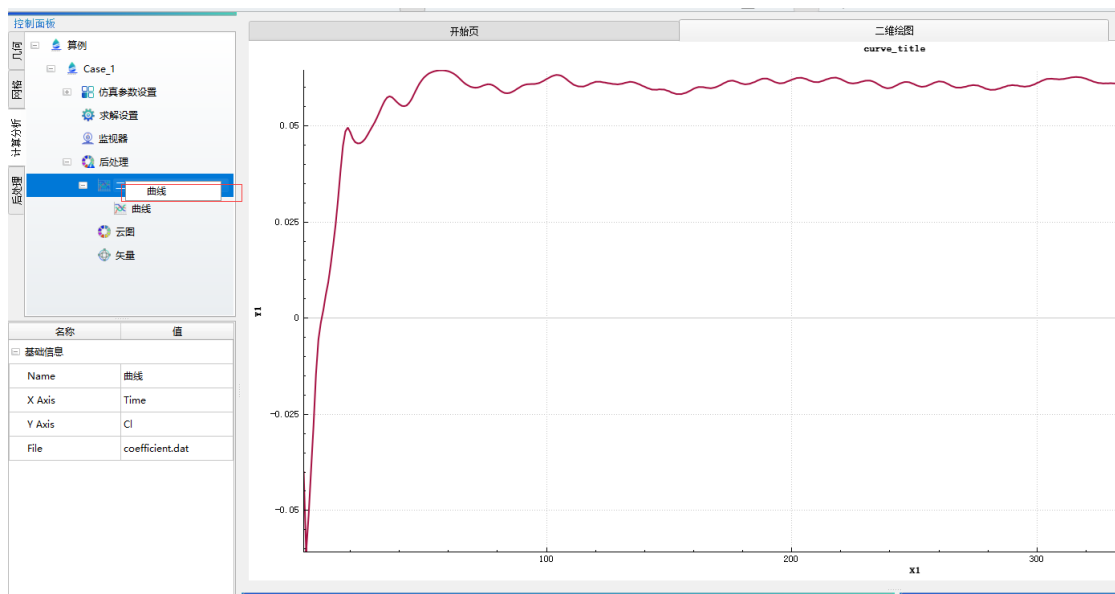


图 45 二维曲线显示

4.3.11 求解器管理

案例求解器设置，点击菜单栏【求解器】-【求解器管理】项，软件将弹出求解器管理界面。



图 46 求解器管理

点击求解器管理界面的 **添加** 按钮，软件将弹出添加求解器界面，按照下图所示添加求解器信息。需要注意的是这里的求解器路径要选择 **Solver.exe** 可执行文件所在的文件路径。点击界面中的 **确认** 按钮完成求解器的添加。



图 47 编辑求解器

- “文件格式”OpenFoamRollDamp 为我们注册的求解器输入文件生成的方法
- ”转换器“OpenFoamFileTrans 为我们求解完成后进行文件转换注册的方法。

此时的求解器管理界面如下图所示，点击界面右上角的  按钮，完成求解器的设置。

4.4 OpenFoam 插件开发

moduleBase	2020-03-05 10:06	文件夹	
ParaClassFactory	2020-03-05 9:33	文件夹	
PluginComplex	2020-03-05 10:06	文件夹	
PluginCustomizer	2020-03-05 10:06	文件夹	
PluginETS	2020-03-05 11:07	文件夹	
PluginHull	2020-03-05 10:06	文件夹	
PluginMachinery	2020-03-05 10:06	文件夹	
PluginManager	2020-03-05 10:06	文件夹	
PluginModel	2020-03-05 10:07	文件夹	
PluginMotorBike	2020-03-10 9:46	文件夹	
PostWidgets	2020-03-05 10:06	文件夹	
ProjectTree	2020-03-10 17:42	文件夹	
ProjectTreeExtend	2020-03-05 10:06	文件夹	
python	2020-03-05 10:06	文件夹	
qrc	2020-03-05 9:28	文件夹	
SelfDefObject	2020-03-05 10:06	文件夹	
settings	2020-03-05 10:06	文件夹	
SolutionDataIO	2020-03-05 9:32	文件夹	
SolverControl	2020-03-05 10:06	文件夹	
XGenerateReport	2020-03-05 10:06	文件夹	
cgns.pri	2020-03-05 9:29	Qt Project Includ...	1 KB
Copy_DLLs.bat	2020-03-05 9:28	Windows 批处理...	22 KB
Copy_Pys.bat	2020-03-05 9:28	Windows 批处理...	1 KB
Copy_Pys.sh	2020-03-05 9:28	SH 文件	1 KB
Copy_SOs.sh	2020-03-05 9:28	SH 文件	17 KB
Create_Linux64_Project.sh	2020-03-05 9:28	SH 文件	2 KB
Create_X64_Project.bat	2020-03-05 9:28	Windows 批处理...	3 KB
FastCAE.sdf	2020-03-10 17:42	SQL Server Com...	115,456 KB
FastCAE.sln	2020-03-05 11:21	Microsoft Visual...	34 KB
FastCAE.v12.suo	2020-03-10 17:42	Visual Studio Sol...	117 KB
log.docx	2020-03-05 9:29	Microsoft Word ...	36 KB
occ.pri	2020-03-05 9:29	Qt Project Includ...	22 KB
PluginMotorBike.zip	2020-03-10 17:40	360压缩 ZIP 文件	849 KB
python.pri	2020-03-05 9:29	Qt Project Includ...	1 KB
ReadMe.txt	2020-03-05 9:29	文本文档	2 KB
Run_MSVC.bat	2020-03-05 9:29	Windows 批处理...	1 KB
Translate.bat	2020-03-05 9:28	Windows 批处理...	2 KB
vtk.pri	2020-03-05 9:29	Qt Project Includ...	34 KB

图 48 插件代码存放目录

项目文件夹名为 **PluginMotorBike**，与 **FastCAE** 其他项目放到相同路径，此项目源码可从网站下载，编译后做为 **FastCAE** 的一个插件，实现求解器参数文件生成和求解计算后文件的转换。

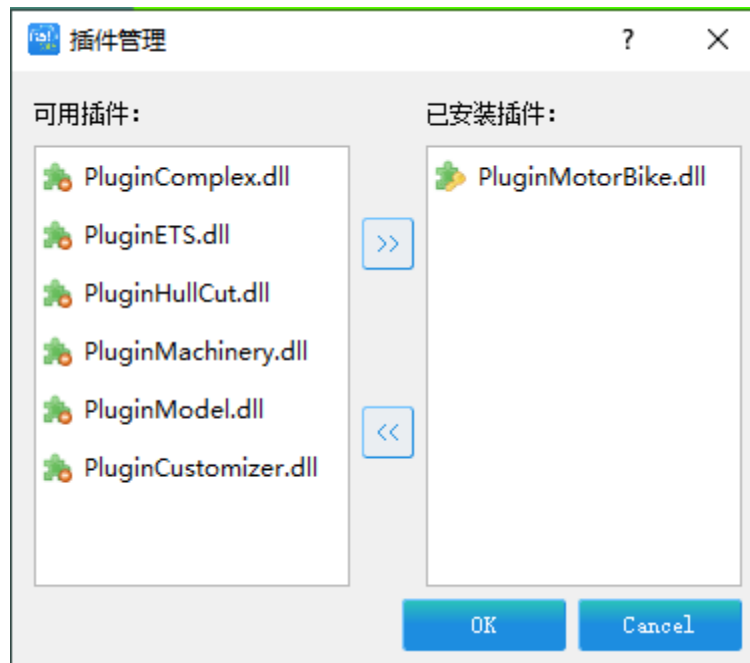


图 49 加载插件

4.4.1 头文件说明

我们的插件继承自插件基类 `PluginBase`，因此基类的纯虚函数需要在我们的插件类中实现。这里我们把我们的插件类名字命名为 `PluginOpenFoamExtend`。此时类所在的头文件 `PluginOpenFoamExtend.h` 内容如下：

```
namespace Plugins
{
    static GUI::MainWindow* mMainWindow;
    class OPENFOAMPLUGINAPI PluginOpenFoamExtend:public PluginBase
    {
        Q_OBJECT
    public:
        PluginOpenFoamExtend(GUI::MainWindow* m);
        PluginOpenFoamExtend();
        //加载插件
        bool install() override;
        //卸载插件
        bool uninstall() override;
    public:
        static int invalue;
```



```

        static GUI::MainWindow* getMainWindow();
private:
    //发送控制台消息
    void sendMessageToConsole(QString hint);
signals:
    void treePrintMessageToMessageWindowSignal(ModuleBase::Message message);
};
}
extern "C"
{
    void OPENFOAMPLUGINAPI Register(GUI::MainWindow*m,
    QList<Plugins::PluginBase*>*plugins);
    void showmsg(QString inputtext);
    //求解器文件写出，给求解器使用
    bool OPENFOAMPLUGINAPI WriteOut(QString path, ModelData::ModelDataBase*d);
    //求解完成后的文件转换
    bool OPENFOAMPLUGINAPI transfer(QString);
    //复制字典文件到case目录下
    bool OPENFOAMPLUGINAPI copyDirectoryFiles(const QString &fromDir, const QString
&toDir, bool coverFileIfExist);
    //求解计算完成后复制后处理文件到case下目录
    bool OPENFOAMPLUGINAPI transfileToDest(const QString &fromDir, const QString &toDir,
int type);
    //发送消息到控制台
    void OPENFOAMPLUGINAPI sendMessageToConsole(QString hintInfo);
    //生成openfoam字典里读取的文本文件
    bool OPENFOAMPLUGINAPI writeinputText(QString filename,
ModelData::ModelDataBase*d);
}

```

4.4.2 CPP 文件说明

此时类所在的 CPP 文件 `PluginOpenFoamExtend.cpp` 内容如下：

```

namespace Plugins
{
    PluginOpenFoamExtend::PluginOpenFoamExtend(GUI::MainWindow* m)
    {
        mMainWindow = m;
        _describe = "OpenFoam plugin";
    }

    bool PluginOpenFoamExtend::install()

```

```

{
    //注册求解器输入文件生成函数
    IO::IOConfigure::RegisterInputFile("OpenFoamRollDamp", WriteOut);
    //注册求解完成后文件转换函数
    IO::IOConfigure::RegisterOutputTransfer("OpenFoamFileTrans", transfer);

    return true;
}

bool PluginOpenFoamExtend::uninstall()
{
    IO::IOConfigure::RemoveInputFile("OpenFoamRollDamp");
    return true;
}

GUI::MainWindow* PluginOpenFoamExtend::getMainWindow()
{
    return mMainWindow;
}
}

void Register(GUI::MainWindow* m, QList<Plugins::PluginBase*>* ps)
{
    Plugins::PluginBase* p = new Plugins::PluginOpenFoamExtend(m);
    ps->append(p);
}

bool WriteOut(QString path, ModelData::ModelDataBase* d)
{
    //生成输入文件
    return true;
}

bool transfer(QString path)
{
    //求解完成后把后处理vtk文件复制到result目录下
}

bool OPENFOAMPLUGINAPI copyDirectoryFiles(const QString &fromDir, const QString &toDir,
bool coverFileIfExist)
{
    //新建算例时复制字典文件到case目录下
}

void OPENFOAMPLUGINAPI sendMessageToConsole(QString hintInfo)

```

```

{
    //发送消息到控制台
    GUI::MainWindow* mainwindow=nullptr;
    mainwindow = Plugins::PluginOpenFoamExtend::getMainWindow();
    if (mainwindow == nullptr) return;
    ModuleBase::Message message;
    message.type = ModuleBase::MessageType::Warning_Message;
    message.message = hintInfo;
    emit mainwindow->printMessageToMessageWindow(message);
}

bool OPENFOAMPLUGINAPI writeinputText(QString filename, ModelData::ModelDataBase*d)
{
    //生成关联字典文件的输入参数文件
}

bool OPENFOAMPLUGINAPI transfileToDest(const QString &fromDir, const QString &toDir, int
type)
{
    //复制后处理文件到指定目录
}

```

在基类 `PluginBase` 中有一个 `Protected` 成员变量 `QString _describe`，它是用于描述当前插件信息的字符串变量。这里我们在 `FluentPlugin` 类的构造函数中对其赋值。

```

_describe = "OpenFoam plugin";

```

4.4.3 插件主要函数说明

当前 FastCAE2.0 插件支持的输入输出扩展函数包括：

```

//注册写出的后缀与方法
static void RegisterInputFile(QString suffix, WRITEINPFILE fun);

//注册结果文件转换方法
static void RegisterOutputTransfer(QString name, TRANSFEROUTFILE fun);

```

需要注意的是 `RegisterInputFile` 函数注册完成后，当在求解器设置成我们所扩展的格式时，该函数会在求解器求解之前进行调用。而

RegisterOutputTransfer 函数注册完成后，会在完成求解器求解生成结果文件后调用。

根据我们的需求我们需要在**求解器求解之前**完成求解器输入文件 **initparameter** 生成。因此我们选择 **RegisterInputFile** 方法作为我们拓展的方法。因此我们在头文件中 **C** 函数部分添加：

```
bool OPENFOAMPLUGINAPI WriteOut(QString path, ModelData::ModelDataBase*d);
```

在 **CPP** 文件中添加：

```
bool WriteOut(QString path, ModelData::ModelDataBase* d)
{
    return true;
}
```

并在 **WriteOut** 函数中添加相应的功能代码，具体代码可以参见本段落最终头文件和 **CPP** 文件全部内容部分。

根据我们的需求我们需要在**求解**完成后把结果文件复制到固定目录下用于后处理展示，因此我们选择 **RegisterOutputTransfer** 方法作为我们拓展的方法。因此我们在头文件中 **C** 函数部分添加：

```
bool OPENFOAMPLUGINAPI transfer(QString);
```

在 **CPP** 文件中添加：

```
bool transfer(QString path)
{
    QStringList rowList;
    QString sourcePath = path + "/postProcessing/cuttingPlane";
    QString destPath = path + "/Result/CuttingPlane/";
    if (!transfileToDest(sourcePath, destPath, 0))
    {
        sendMessageToConsole(QStringLiteral("从PostProcessing/cuttingPlane目录复制文件到Result/cuttingPlane出错"));
        return false;
    }
    sourcePath = path + "/postProcessing/sets/streamLines/";
```

```

destPath = path + "/Result/StreamLines/";
if (!transfileToDest(sourcePath, destPath,1))
{
    sendMessageToConsole(QStringLiteral("从PostProcessing/sets/streamLines目录复制文件到Result/cuttingPlane出错"));
    return false;
}
QString fileName = path + "/postProcessing/forceCoeffs1/0/coefficient.dat";
QFile file_y(fileName);
if (!file_y.exists())
{
    sendMessageToConsole(QStringLiteral("postProcessing/forceCoeffs1/0/目录下, 不存在coefficient.dat文件转换出错"));
    return false;
}

if (!file_y.open(QIODevice::ReadOnly))
    return false;
rowList.clear();
QString lineStr;
while (!file_y.atEnd())
{
    lineStr = file_y.readLine().trimmed();
    if (lineStr != "")
    {
        if (!lineStr.contains("# Time"))
        {
            continue;
        }
        lineStr = "Time      Cm      Cd      Cl      Cl(f)      Cl(r)";
        rowList.append(lineStr);
        break;
    }
}
while (!file_y.atEnd())
{
    lineStr = file_y.readLine();
    lineStr = lineStr.replace(" ", " ");
    rowList.append(lineStr);
}
QString curFileName;
curFileName = path + "/Result/coefficient.dat";
QFile file(curFileName);
if (!file.open(QIODevice::WriteOnly | QIODevice::Text))return false;

```

```

    QTextStream stream(&file);
    for (int i = 0; i < rowList.count(); i++)
    {
        stream << rowList.at(i) << endl;
    }
    return true;
}

```

并在 **transfer** 函数中添加相应的功能代码，具体代码可以参见本段落最终头文件和 **CPP** 文件全部内容部分。

4.4.4 插件全部代码

插件最终头文件内容：

```

#ifndef PLUGINOPENFOAMEXTEND_H
#define PLUGINOPENFOAMEXTEND_H

#include "OpenFoamPluginAPI.h"
#include "PluginManager/pluginBase.h"
#include "DataProperty/modelTreeItemType.h"
#include "mainWindow/mainWindow.h"
#include "DataProperty/ParameterTable.h"
#include "DataProperty/ParameterSelectable.h"
#include "DataProperty/ParameterInt.h"
#include "DataProperty/ParameterBool.h"
#include "DataProperty/ParameterDouble.h"

namespace IO
{
    class SolverIO;
}
class IOConfigure;
namespace ModelData
{
    class ModelDataBase;
}
namespace GUI
{
    class MainWindow;
}
namespace Plugins
{

```

```

static GUI::MainWindow* mMainWindow;
class OPENFOAMPLUGINAPI PluginOpenFoamExtend:public PluginBase
{
    Q_OBJECT
public:
    PluginOpenFoamExtend(GUI::MainWindow* m);
    PluginOpenFoamExtend();
    //加载插件
    bool install() override;
    //卸载插件
    bool uninstall() override;
public:
    static int invalue;
    static GUI::MainWindow* getMainWindow();
private:
    //发送控制台消息
    void sendMessageToConsole(QString hint);
signals:
    void treePrintMessageToMessageWindowSignal(ModuleBase::Message message);
};
}
extern "C"
{
    void OPENFOAMPLUGINAPI Register(GUI::MainWindow*m,
    QList<Plugins::PluginBase*>*plugins);
    void showmsg(QString inputtext);
    //求解器文件写出，给求解器使用
    bool OPENFOAMPLUGINAPI WriteOut(QString path, ModelData::ModelDataBase*d);
    //求解完成后的文件转换
    bool OPENFOAMPLUGINAPI transfer(QString);
    //复制字典文件到case目录下
    bool OPENFOAMPLUGINAPI copyDirectoryFiles(const QString &fromDir, const QString
&toDir, bool coverFileIfExist);
    //求解计算完成后复制后处理文件到case下目录
    bool OPENFOAMPLUGINAPI transfileToDest(const QString &fromDir, const QString &toDir,
int type);
    //发送消息到控制台
    void OPENFOAMPLUGINAPI sendMessageToConsole(QString hintInfo);
    //生成openfoam字典里读取的文本文件
    bool OPENFOAMPLUGINAPI writeinputText(QString filename,
ModelData::ModelDataBase*d);
    QString getParameterValue(QString paraname, ModelData::ModelDataBase*d);
    DataProperty::ParameterTable* getParameterTable(QString paraname,
ModelData::ModelDataBase*d);

```

```
DataProperty::ParameterSelectable* getParameterSelectable(QString paraname,  
ModelData::ModelDataBase* d);
```

```
}  
#endif // PLUGINOPENFOAMEXTEND_H
```

插件最终 CPP 文件内容:

```
#include "PluginOpenFoamExtend.h"  
class IOConfigure;  
#include <QDebug>  
#include "mainWindow/mainWindow.h"  
#include "ModelData/modelDataFactory.h"  
#include "ModelData/modelDataSingleton.h"  
#include "ModelData/modelDataBase.h"  
#include "IO/IOConfig.h"  
#include <QDir>  
#include <QApplication>  
#include <QFile>  
#include <vtkSTLWriter.h>  
#include <vtkSmartPointer.h>  
#include <QTextCodec>  
#include "meshData/meshSingleton.h"  
#include "meshData/meshKernal.h"  
#include <vtkDataSet.h>  
#include <QTextStream>  
#include <QTextCodec>  
#include "IO/SolverIO.h"  
#include <QtMath>  
#include <QMessageBox>  
namespace GUI  
{  
    class MainWindow;  
}  
namespace Plugins  
{  
  
    PluginOpenFoamExtend::PluginOpenFoamExtend(GUI::MainWindow* m)  
    {  
        mMainWindow = m;  
        _describe = "OpenFoam plugin";  
    }  
  
    bool PluginOpenFoamExtend::install()  
    {
```



```

        IO::IOConfigure::RegisterInputFile("OpenFoamRollDamp", WriteOut);
        IO::IOConfigure::RegisterOutputTransfer("OpenFoamFileTrans", transfer);

        return true;
    }

    bool PluginOpenFoamExtend::uninstall()
    {
        IO::IOConfigure::RemoveInputFile("OpenFoamRollDamp");
        IO::IOConfigure::RemoveInputFile("OpenFoamFileTrans");
        return true;
    }

    GUI::MainWindow* PluginOpenFoamExtend::getMainWindow()
    {
        return mMainWindow;
    }
}

void Register(GUI::MainWindow* m, QList<Plugins::PluginBase*>* ps)
{
    //Plugins::PluginOpenFoamExtend::invalue = 1;
    Plugins::PluginBase* p = new Plugins::PluginOpenFoamExtend(m);
    ps->append(p);
}

bool WriteOut(QString path, ModelData::ModelDataBase* d)
{
    transfer(path);
    QString filename;
    QString casepath;
    casepath = d->getPath() + "/";
    qDebug() << d->getName();
    qDebug() << "writeTest";
    QString binpath = QCoreApplication::applicationDirPath();
    QDir dir;
    QString resultpath;
    resultpath = casepath + "Result/";
    if (!dir.exists(resultpath))
    {
        bool res = dir.mkpath(resultpath);
    }
    if (!copyDirectoryFiles(binpath + "../solver/dictionary", casepath, true))
    {
        sendMessageToConsole(QObject::tr("Copy dictionary fail,please check!"));
    }
}

```

```

        return false;
    }
    if (!writeInputText((casepath + "initparameter"),d))
    {
        return false;
    }
    return true;
}
bool transfer(QString path)
{
    QStringList rowList;
    QString sourcePath = path + "/postProcessing/cuttingPlane";
    QString destPath = path + "/Result/CuttingPlane/";
    if (!transfileToDest(sourcePath, destPath, 0))
    {
        sendMessageToConsole(QStringLiteral("从PostProcessing/cuttingPlane目录复制文件到
        Result/cuttingPlane出错"));
        return false;
    }
    sourcePath = path + "/postProcessing/sets/streamLines/";
    destPath = path + "/Result/StreamLines/";
    if (!transfileToDest(sourcePath, destPath,1))
    {
        sendMessageToConsole(
        QStringLiteral("从PostProcessing/sets/streamLines目录复制文件到Result/cuttingPlane出错"));
        return false;
    }
    QString fileName = path + "/postProcessing/forceCoeffs1/0/coefficient.dat";
    QFile file_y(fileName);

    if (!file_y.exists())
    {
        sendMessageToConsole(QStringLiteral("postProcessing/forceCoeffs1/0/目
        录下，不存在coefficient.dat文件转换出错"));
        return false;
    }
    if (!file_y.open(QIODevice::ReadOnly))
        return false;
    rowList.clear();
    QString lineStr;
    while (!file_y.atEnd())
    {
        lineStr = file_y.readLine().trimmed();
        if (lineStr != "")

```

```

    {
        if (!lineStr.contains("# Time"))
        {
            continue;
        }
        lineStr = "Time      Cm      Cd      Cl      Cl(f)      Cl(r)";
        rowList.append(lineStr);
        break;
    }
}
while (!file_y.atEnd())
{
    lineStr = file_y.readLine();
    lineStr = lineStr.replace("      ", " ");
    rowList.append(lineStr);
}
QString curFileName;
curFileName = path + "/Result/coefficient.dat";
QFile file(curFileName);
if (!file.open(QIODevice::WriteOnly | QIODevice::Text))return false;
QTextStream stream(&file);
for (int i = 0; i < rowList.count(); i++)
{
    stream << rowList.at(i) << endl;
}
return true;
}

```

```

bool OPENFOAMPLUGINAPI copyDirectoryFiles(const QString &fromDir, const QString &toDir,
bool coverFileIfExist)

```

```

{
    QDir sourceDir(fromDir);
    QDir targetDir(toDir);
    if (!targetDir.exists()){    /**< 如果目标目录不存在，则进行创建 */
        if (!targetDir.mkdir(targetDir.absolutePath()))
            return false;
    }
    QFileInfoList fileInfoList = sourceDir.entryInfoList();
    foreach(QFileInfo fileInfo, fileInfoList){
        if (fileInfo.fileName() == "." || fileInfo.fileName() == "..")
            continue;
        if (fileInfo.isDir())
        {
            /**< 当为目录时，递归的进行copy */
            if (!copyDirectoryFiles(fileInfo.filePath(),

```

```

        targetDir.filePath(fileInfo.fileName()), coverFileIfExist))
        return false;
    }
    else{
        /**< 当允许覆盖操作时，将旧文件进行删除操作 */

        if (coverFileIfExist && targetDir.exists(fileInfo.fileName())){
            targetDir.remove(fileInfo.fileName());
        }
        /// 进行文件copy
        if (!QFile::copy(fileInfo.filePath(),
            targetDir.filePath(fileInfo.fileName()))){
            return false;
        }
    }
}

return true;
}

```

```

void OPENFOAMPLUGINAPI sendMessageToConsole(QString hintInfo)
{
    GUI::MainWindow* mainwindow=nullptr;
    mainwindow = Plugins::PluginOpenFoamExtend::getMainWindow();
    if (mainwindow == nullptr) return;
    ModuleBase::Message message;
    message.type = ModuleBase::MessageType::Warning_Message;
    message.message = hintInfo;
    emit mainwindow->printMessageToMessageWindow(message);
}

```

```

bool OPENFOAMPLUGINAPI writeinputText(QString filename, ModelData::ModelDataBase*d)
{
    QString splitchar = " ";
    QString paravalue;
    QStringList list;
    QString startTime;
    QString endTime;

    paravalue = getParameterValue(QStringLiteral("开始步"), d);

    list.append(QStringLiteral("starttime%1%2").arg(splitchar).arg(paravalue));
    paravalue = getParameterValue(QStringLiteral("结束步"), d);

```

```
list.append(QStringLiteral("endtime%1%2").arg(splitchar).arg(paravalue));
```

```
paravalue = getParameterValue(QStringLiteral("求解器名称"), d);
```

```
list.append(QStringLiteral("application%1%2").arg(splitchar).arg(paravalue));
```

```
paravalue = getParameterValue(QStringLiteral("文件生成间隔"), d);
```

```
list.append(QStringLiteral("writeInterval%1%2").arg(splitchar).arg(paravalue));
```

```
//blockmesh
```

```
DataProperty::ParameterTable* pNx = getParameterTable(QStringLiteral("各方向网格数量"),  
d);
```

```
if (pNx != nullptr)
```

```
{
```

```
    if (pNx->getRowCount() > 0 && pNx->getColumnCount() > 2)
```

```
    {
```

```
        list.append(QStringLiteral("xgridnumber%1%2").arg(splitchar).arg(pNx->getValue(0, 0)));
```

```
        list.append(QStringLiteral("ygridnumber%1%2").arg(splitchar).arg(pNx->getValue(0, 1)));
```

```
        list.append(QStringLiteral("zgridnumber%1%2").arg(splitchar).arg(pNx->getValue(0, 2)));
```

```
    }
```

```
}
```

```
int rowcount;
```

```
DataProperty::ParameterTable* pNy = getParameterTable(QStringLiteral("顶点坐标"), d);
```

```
if (pNy != nullptr)
```

```
{
```

```
    rowcount = pNy->getRowCount();
```

```
    for (int i = 0; i < rowcount; i++)
```

```
    {
```

```
        double x, y, z;
```

```
        x = pNy->getValue(i, 0);
```

```
        y = pNy->getValue(i, 1);
```

```
        z = pNy->getValue(i, 2);
```

```
        list.append(QStringLiteral("x%1%2%3").arg(i + 1).arg(splitchar).arg(x));
```

```
        list.append(QStringLiteral("y%1%2%3").arg(i + 1).arg(splitchar).arg(y));
```

```
        list.append(QStringLiteral("z%1%2%3").arg(i + 1).arg(splitchar).arg(z));
```

```
    }
```

```
}
```

```
//decomposepredict
```

```
DataProperty::ParameterTable* px = getParameterTable(QStringLiteral("各方向分解数量"), d);
```

```
if (px != nullptr)
```

```

{
    if (px->getColumnCount()> 2&&px->getRowCount()>0)
    {
        list.append(QStringLiteral("nx%1%2").arg(splitchar).arg(px->getValue(0, 0)));
        list.append(QStringLiteral("ny%1%2").arg(splitchar).arg(px->getValue(0, 1)));
        list.append(QStringLiteral("nz%1%2").arg(splitchar).arg(px->getValue(0, 2)));
    }

}

paravalue = getParameterValue(QStringLiteral("分解域的总数"), d);
list.append(QStringLiteral("core%1%2").arg(splitchar).arg(paravalue));

DataProperty::ParameterSelectable *pMethod = getParameterSelectable(QStringLiteral("分解方法"), d);
if (pMethod != nullptr)
{
    int index = pMethod->getCurrentIndex();
    QString methodstr;
    if (index == 0)
    {
        methodstr = "hierarchical";
    }
    else
    {
        methodstr = "ptscotch";
    }
    list.append(QStringLiteral("decomposemethod%1%2").arg(splitchar).arg(methodstr));
}

//snappyHexMeshDict:
paravalue = getParameterValue(QStringLiteral("是否切割网格"), d);

list.append(QStringLiteral("castellatedMesh%1%2").arg(splitchar).arg(paravalue));

paravalue = getParameterValue(QStringLiteral("是否进行网格贴合"), d);

list.append(QStringLiteral("snap%1%2").arg(splitchar).arg(paravalue));

paravalue = getParameterValue(QStringLiteral("添加网格边界层"), d);

list.append(QStringLiteral("addLayers %1%2").arg(splitchar).arg(paravalue));

paravalue = getParameterValue(QStringLiteral("最小表面细化等级"), d);

```

```

list.append(QStringLiteral("minlevel%1%2").arg(splitchar).arg(paravalue));

paravalue = getParameterValue(QStringLiteral("最大表面细化等级"), d);

list.append(QStringLiteral("maxlevel%1%2").arg(splitchar).arg(paravalue));

paravalue = getParameterValue(QStringLiteral("边界层添加迭代数"), d);

list.append(QStringLiteral("nLayerIter%1%2").arg(splitchar).arg(paravalue));

//controldict

QFile file(filename);
if (!file.open(QIODevice::WriteOnly | QIODevice::Text))return false;
QTextStream stream(&file);
for (int i = 0; i < list.count(); i++)
{
    stream << list.at(i)+";" << endl;
}
return true;
}

QString getParameterValue(QString paraname, ModelData::ModelDataBase*d)
{
    QString t;
    DataProperty::ParameterBase* p = d->getParameterByName(paraname);
    if (p != nullptr)
    {
        if (p->getParaType() == DataProperty::Para_String)
        {
            t= p->valueToString();
        }
        else if (p->getParaType() == DataProperty::Para_Int)
        {
            DataProperty::ParameterInt* pInt=(DataProperty::ParameterInt*)(p);
            t = QString("%1").arg(pInt->getValue());
        }
        else if (p->getParaType() == DataProperty::Para_Double)
        {
            DataProperty::ParameterDouble* pDouble = (DataProperty::ParameterDouble*)(p);
            t = QString("%1").arg(pDouble->getValue());
        }
        else if (p->getParaType() == DataProperty::Para_Bool)

```

```

{
    DataProperty::ParameterBool* pBool = (DataProperty::ParameterBool*)(p);
    return pBool->valueToString();
    if (pBool->getValue())
    {
        t = QString("%1").arg("true");
    }
    else
    {
        t = QString("%1").arg("false");
    }

}

return t;
}
else
{
    return "0.0000";
}
}

```

DataProperty::ParameterTable* getParameterTable(QString paraname,
ModelData::ModelDataBase*d)

```

{
    DataProperty::ParameterTable* result = nullptr;
    DataProperty::ParameterBase* p = d->getParameterByName(paraname);
    if (p != nullptr)
    {
        if (p->getParaType() == DataProperty::Para_Table)
        {
            result = (DataProperty::ParameterTable*)(p);
        }
    }
    return result;
}

```

DataProperty::ParameterSelectable* getParameterSelectable(QString paraname,
ModelData::ModelDataBase* d)

```

{
    DataProperty::ParameterSelectable* result = nullptr;
    DataProperty::ParameterBase* p = d->getParameterByName(paraname);
    if (p != nullptr)
    {
        if (p->getParaType() == DataProperty::Para_Selectable)

```



```

        {
            result = (DataProperty::ParameterSelectable*)(p);
        }
    }
    return result;
}

void showmsg(QString inputtext)
{
    QMessageBox msgBox;
    msgBox.setText(inputtext);
    msgBox.exec();
}

bool OPENFOAMPLUGINAPI transfileToDest(const QString &fromDir, const QString &toDir, int
type)
{
    int i = 0;
    QStringList dirlist;
    QDir sourceDir(fromDir);
    sourceDir.setFilter(QDir::Dirs);
    sourceDir.setSorting(QDir::Name);
    QDir targetDir(toDir);
    if (!targetDir.exists()){    /**< 如果目标目录不存在，则进行创建 */
        if (!targetDir.mkdir(targetDir.absolutePath()))
            return false;
    }

    QString destFileName;
    QFileInfoList fileInfoList = sourceDir.entryInfoList();
    int fileno = 0;
    foreach(QFileInfo fileInfo, fileInfoList)
    {
        if (fileInfo.fileName() == "." || fileInfo.fileName() == "..")
            continue;
        QString dirname = fileInfo.filePath();
        if (fileInfo.isDir())
        {
            QDir fileDir(fileInfo.filePath());

            fileDir.setFilter(QDir::Files);

            QFileInfoList fileList = fileDir.entryInfoList();

```

```

destFileName = "";

for (int j = 0; j < fileList.count(); j++)
{
    if (fileList.at(j).fileName() == "." || fileList.at(j).fileName() == "..") continue;
    if (type == 0)
    {
        if (fileList.at(j).fileName().contains("p_y"))
        {
            destFileName = QString("%1%2%3%4").arg(toDir).arg("p").arg(fileno).arg(".vtk");
            fileno++;
        }
    }
    else
    {
        if (fileList.at(j).fileName().contains("track0_p"))
        {
            destFileName =
                QString("%1%2%3%4").arg(toDir).arg("streamline").arg(fileno).arg(".vtk");
            fileno++;
        }
    }
    if (targetDir.exists(fileList.at(j).fileName()))
    {
        targetDir.remove(fileList.at(j).fileName());
    }
    QString filename;
    filename = fileList.at(j).filePath();
    if (!QFile::copy(fileList.at(j).filePath(), destFileName));
    {
        //return false;
    }
}
}
return true;
}

```

其中 **writeinputText** 函数用于从框架中获取到所有的设置参数生成字典文件需要包含的 **initparameter** 文件。

4.5 软件使用

4.5.1 插件加载

插件的加载，如下图所示：

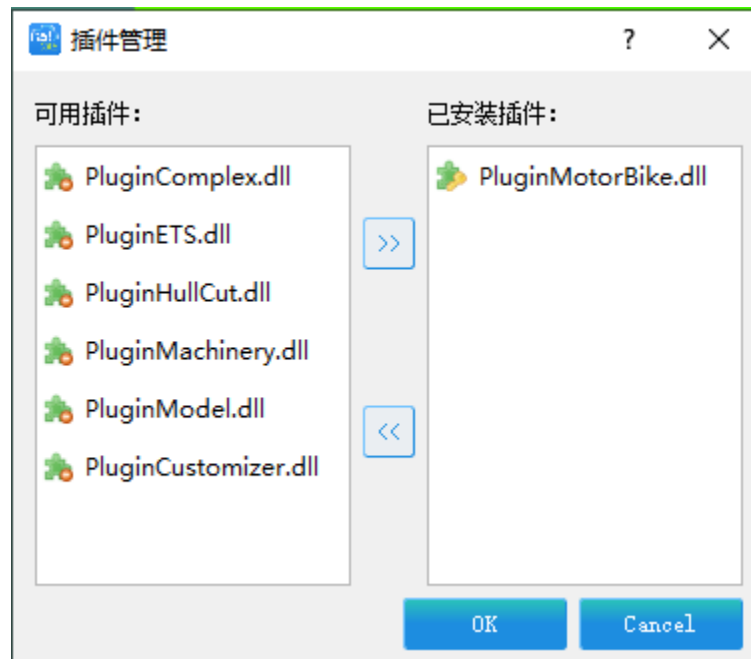


图 50 加载插件

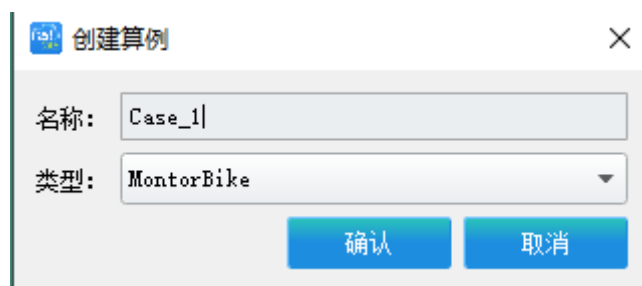
4.5.2 创建算例

鼠标左键点击【控制面板】-【计算分析】-【算例】节点，点击鼠标右键软件将弹出【算例】节点的右键菜单。



图 51 创建算例

1. 点击【算例】节点弹出的右键菜单中的【创建算例】菜单项，软件会弹出创建算例对话框。这里我们使用默认的名称，直接点击  按钮完成算例的创建。如果有需要用户可以输入其他的算例名称。



2. 此时算例下的树形结构如下图所示。点击【算例】下的【仿真参数设置】节点，我们可以看到仿真参数设置节点下的定制的参数。用户可以根据需求，自行调整参数值。

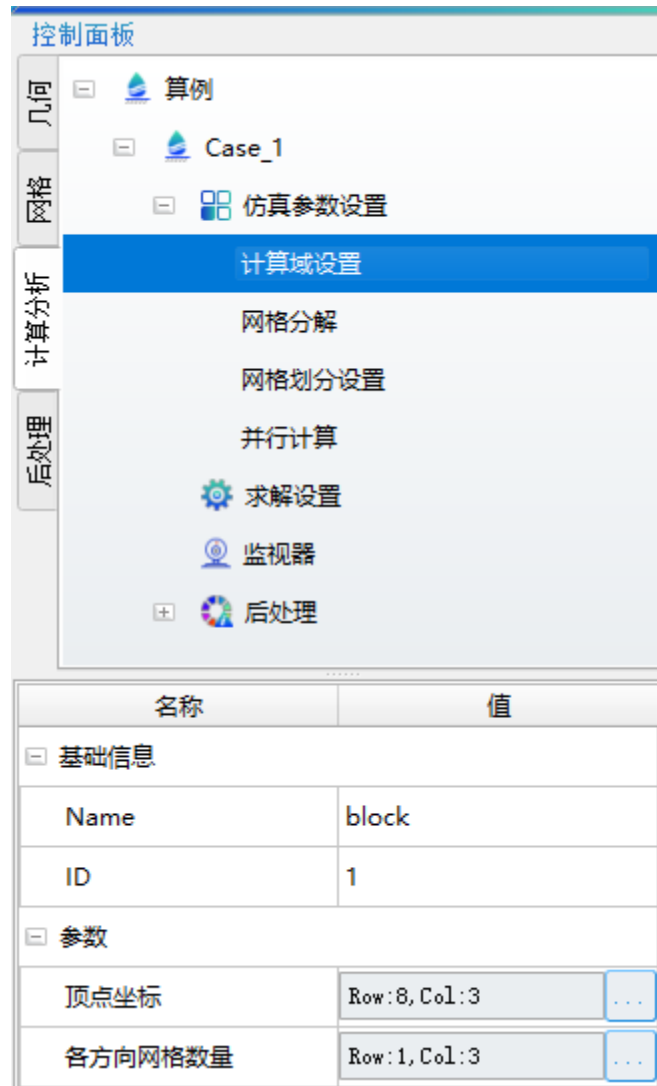


图 52 编辑参数

3. 点击【算例】下的【求解设置】节点，我们可以看到求解设置节点下的定制的参数。用户可以根据需求，自行调整参数值。



图 53 编译求解参数

4.5.3 求解计算

点击工具栏的  按键，软件将启动求解。

在求解器完成后用户可以按照生成 Python 脚本方式加载后处理结果文件进行显示。

4.5.4 后处理展示

4.5.4.1 结果云图展示

本算例求解完成后，后处理文件默认保存在 `postProcessing` 目录下，`cuttingPlane` 和 `sets` 保存 VTK 结果文件，`forceCoeffs1` 保存二维曲线文件，如下图所示：

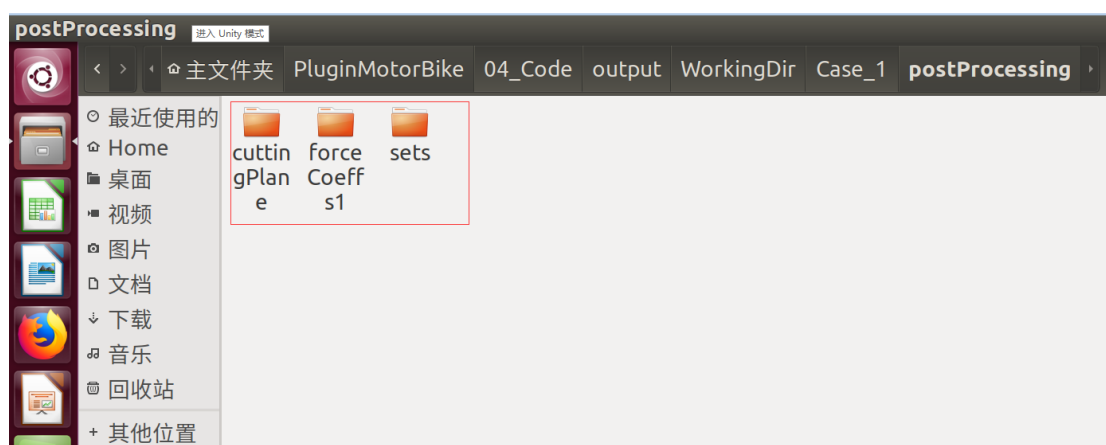


图 54 OpenFoam 后处理保存路径

由于这种目录结构不适合 **FastCAE** 集中动画展示，通过插件的 `transfer` 函数求解完成后将相关文件复制到算例路径下 **Result** 目录下，如下图所示：

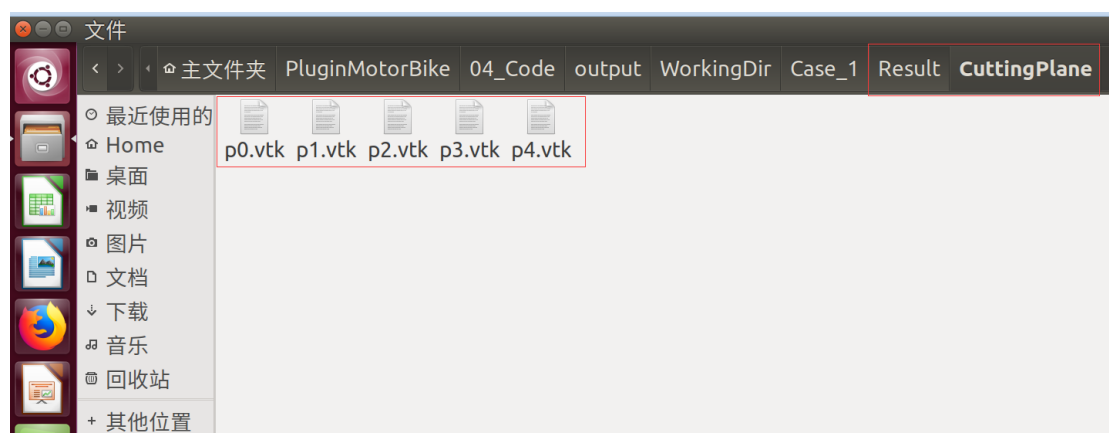


图 55 摩托结果文件

code > work > Project > VtkFrame > OpenFoam > FastCaeMotorBike > 04_Code > output > WorkingDir > Case_1 > Result > StreamLines				
名称	修改日期	类型	大小	
streamline0.vtk	2020-03-06 17:31	VTK 文件	963 KB	
streamline1.vtk	2020-03-06 17:39	VTK 文件	1,348 KB	
streamline2.vtk	2020-03-06 17:46	VTK 文件	988 KB	
streamline3.vtk	2020-03-06 17:53	VTK 文件	1,028 KB	
streamline4.vtk	2020-03-06 18:00	VTK 文件	981 KB	

图 56 流线结果文件

新建算例后，我们打开后处理菜单->三维渲染窗体，点击“打开”，
如下图所示：

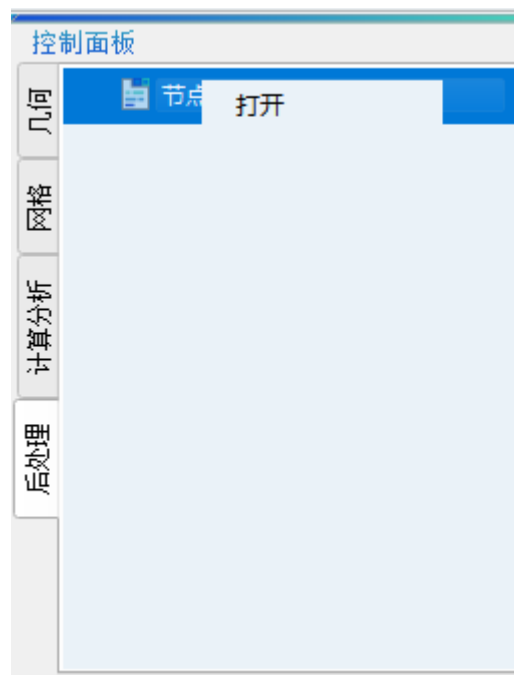


图 57 打开文件夹

选择一组文件加载到树型节点：

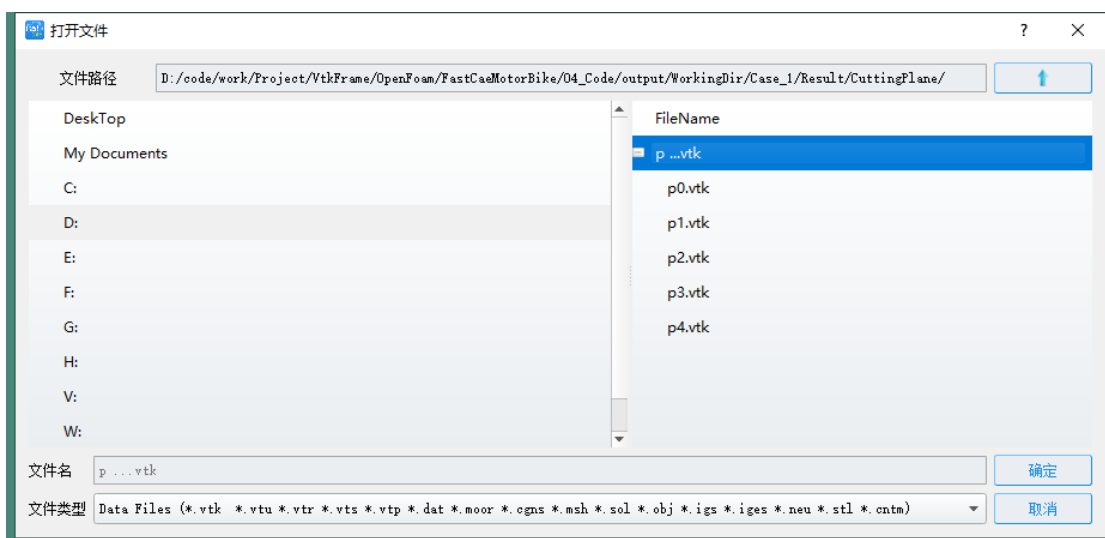


图 58 加载网格文件

再选择一组文件加到树型节点：

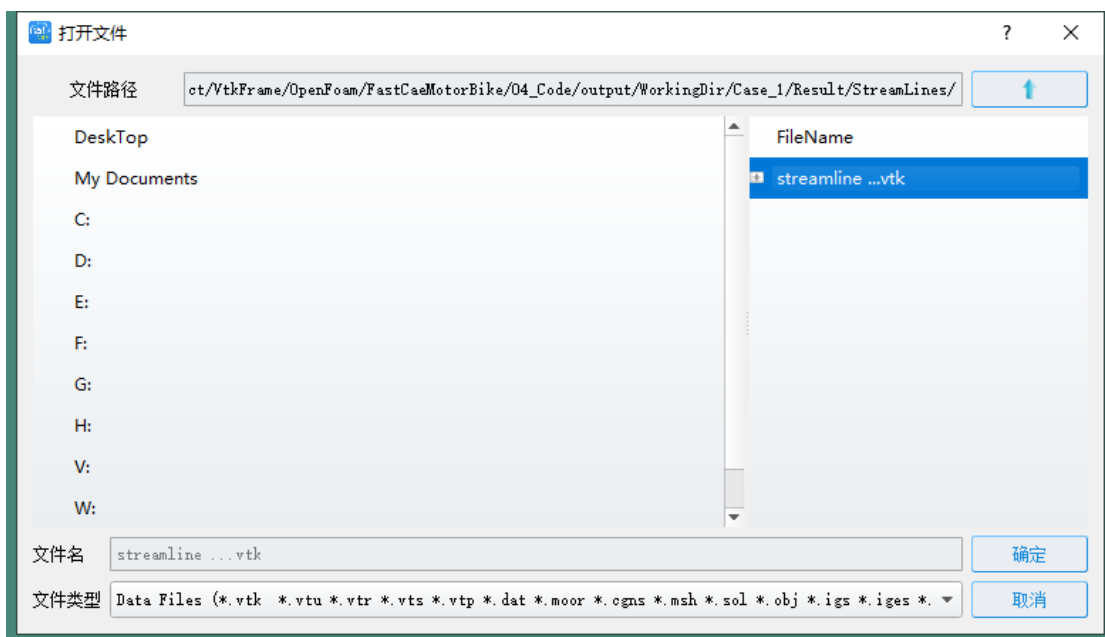


图 59 加载另一组结果文件

点击“应用”按钮，显示出图形。

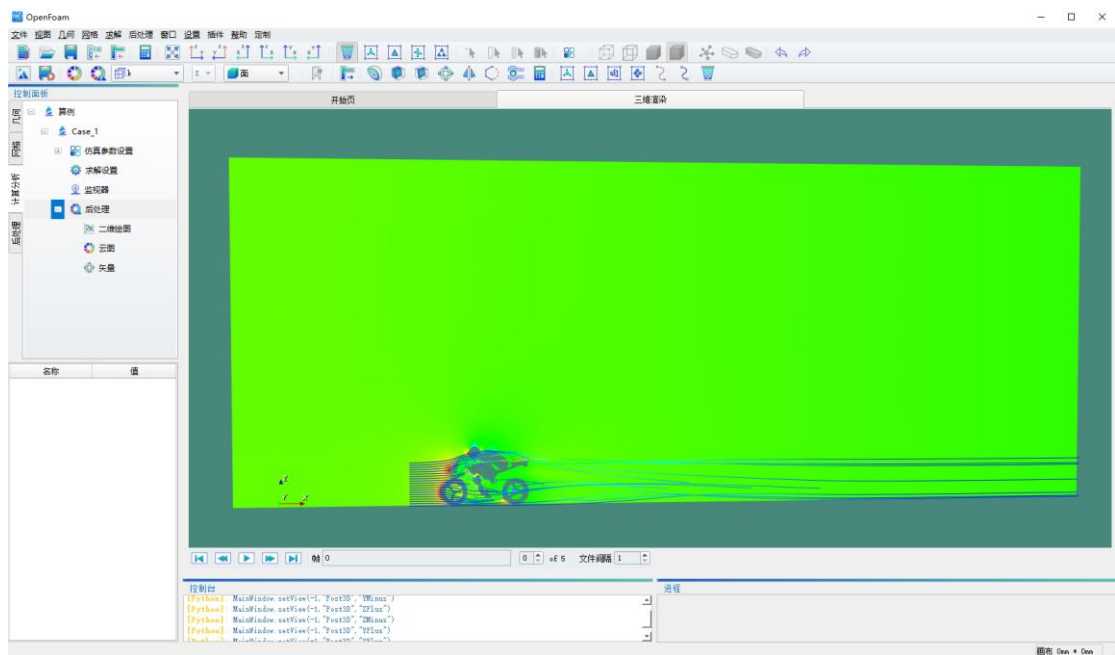


图 60 结果云图展示

4.5.4.2 二维曲线展示

在“二维绘图”节点上单击鼠标右键，显示出二维曲线，如下图所示：



图 61 显示二维曲线操作

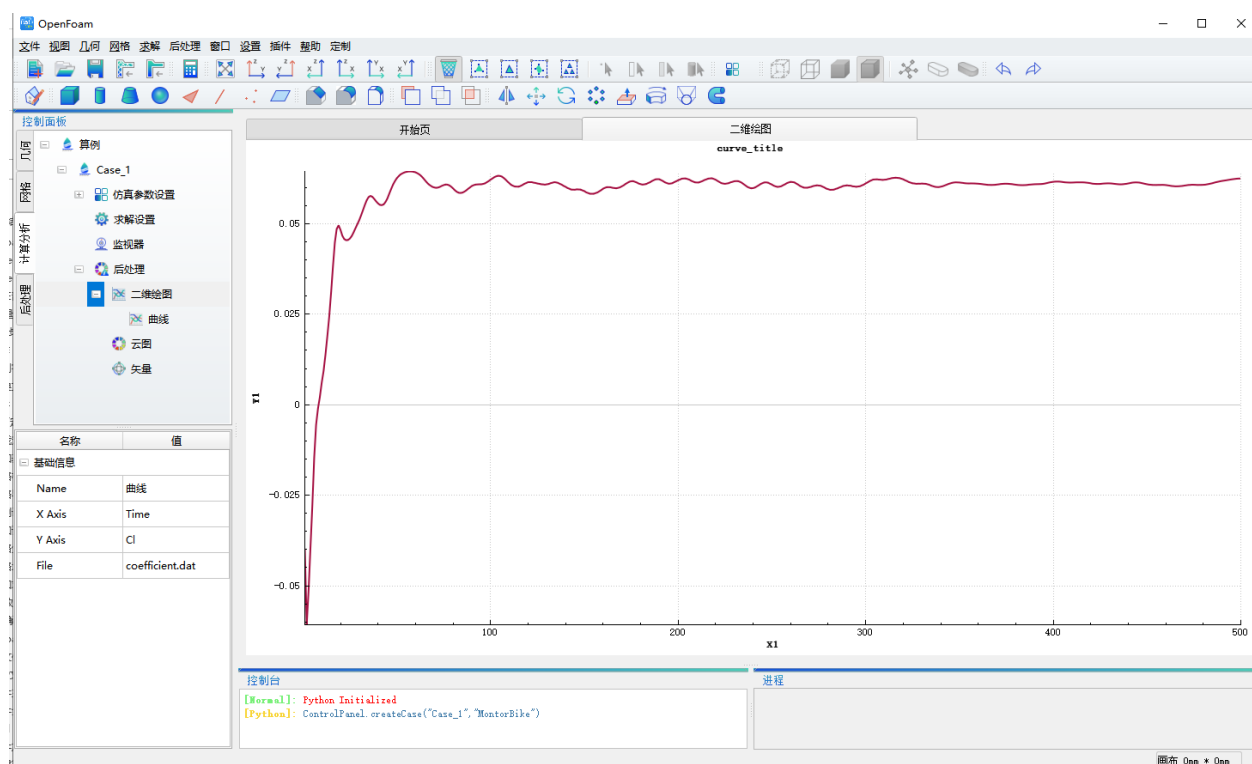


图 62 二维曲线